# Rechnertechnologie

Prof. Dr.-Ing. Franz Brümmer

Sommersemester 2007

Hochschule
Ravensburg–Weingarten
Technik | Wirtschaft | Sozialwesen

# Inhaltsverzeichnis


**1 Einleitung** **4**
1.1 Entwicklungsstufen . . . 5
1.2 Prozessorgattungen . . . 6
1.3 Rechnerarchitektur . . . 7
1.4 Entwurfskriterien der Rechnerarchitektur . . . 9
1.5 Darstellung von Zeichen und Zahlen . . . 10

**2 Prozessorinterne Speicher** **16**
2.1 Elektronische Speicher . . . 16
2.2 Adressierungsarten . . . 20
2.3 Adressdekodierung . . . 23

**3 Sekundäre Speicher** **25**
3.1 Sekundäre Speicher . . . 25
3.2 Festplattenspeicher . . . 25
3.3 CD/DVD . . . 29

**4 Bussysteme und Schnittstellen** **34**
4.1 Serielle Datenübertragung (Com-Port) . . . 34
4.2 Serielle Datenübertragung (USB) . . . 37
4.3 Parallele Datenübertragung . . . 41
4.4 Schnittstellen Motherboard . . . 42

**5 Einführung in die 8086-Architektur** **44**
5.1 Registermodell der 8086-Architektur . . . 44
5.2 Maschinenbefehlssatz des Pentium . . . 46

**6 8086-Architektur im Pentium** **53**
6.1 Begrenzungen der 8086-Architektur . . . 53
6.2 Erweiterungen der 8086-Architektur . . . 55

**7 Assembler und Hochsprache** **60**
7.1 Einordnung der Assemblerprogrammierung . . . 60
7.2 Parameterübergabe . . . 61
7.3 Werkzeuge zur Programmentwicklung . . . 63

**8 Übungen** **66**
8.1 Aufgaben zum Kapitel Einleitung . . . 66
8.2 Aufgaben zu den Kapiteln Speicher . . . 67
8.3 Aufgaben Bussystem und Schnittstellen . . . 67
8.4 Aufgaben zur 8086-Architektur . . . 67

**9 Lösungen zu Übungsaufgaben** **69**
9.1 Aufgaben zum Kapitel Einleitung . . . . . 69
9.2 Aufgaben zu den Kapiteln Speicher . . . . . 70
9.3 Aufgaben Bussystem und Schnittstellen . . . . . 71
9.4 Aufgaben zur 8086-Architektur . . . . . 72

**Literaturverzeichnis** **73**

# Kapitel 1

# Einleitung

Die Vorlesung Rechnertechnologie ist vorgesehen für Studierende der Elektrotechnik, der Angewandten Informatik und der Wirtschaftsinformatik. Nach einem einführenden Teil, in dem ein kurzer Überblick über den Entwurf von Rechnersystemen gegeben wird erfolgt eine Erläuterung von Grundsätzen zum Aufbau von Rechnern (Rechnerarchitektur). Abgeschlossen wird der einleitende Teil der Vorlesung mit einer Darstellung unterschiedlicher Möglichkeiten der Präsentation von Zeichen und Zahlen im Rechner.
Ein weiteres wichtiges Kapitel ist der Aufbau, die Funktionsweise und die Verwendung von unterschiedlichen Speichern in einem Rechnersystem. In der Vorlesung werden elektronische Speicher, magnetische Speicher (Harddisc) und optische Speichermedien vorgestellt. Deren typische Verhaltensweisen, Aufbau und Ansteuerschaltungen werden werden erläutert.
Zur Integration verschiedenster Komponenten innerhalb eines Rechners, sowie der Abschluß externer Komponenten an den Rechner erfordert den Einsatz von Bussystemen. Weiterhin charakterisieren die Bussysteme die Architektur und nicht zuletzt die Leistungsfähigkeit eines Gesamtsystems. Es ist daher wichtig Funktionsweisen von Schnittstellen und Bussystem einschätzen zu können. Innerhalb der Vorlesung werden Mechanismen von seriellen und parallen Datenübertragungen erläutert.
Nach den mehr schaltungsbezogenen ersten Teil der Vorlesung steht im zweiten Teil die Programmierung im Vordergrund. In diesem Teil erfolgt zunächst eine Darstellung des Aufbaus eines Prozessors am Beispiel des Pentiums. Das Registermodell mit seinen Beschränkungen in der 8086-Architektur werden erläutert. Diesen Ausführungen schließen sich die Erweiterungen im Pentium an.
Die eigentliche Programmierung startet mit einer Einführung in die Assemblerprogrammierung. Hierzu ist es notwendig einen Überblick über den Instruction Set des Pentiums zu bekommen. In jeweils kleinen Programmen werden die einzelen Gebiete der Assemblerprogrammierung vorgestellt und eingeübt. Den Abschluss findet dieser Teil der Vorlesung mit der Darstellung der Verknüpfung von Assemblerprogrammen mit C-Funktionen. Dies erfordert eine detailierte Betrachtung der Parameterübergabe zwischen C-Funktionen und Assemberprogrammen.
Mit einem abschließende Betrachtung von Werkzeugen die für die Entwicklung von Programmen zum Einsatz kommen schließt diese Vorlesung.

## 1.1 Entwicklungsstufen

**Entwicklung digitaler Rechner (programmgesteuerte Rechner)**

- Beginn ca. 1940 (1941 Zuse Z3, 1944 Aiken MARK I)
- Einteilung der digitalen Rechner in Generationen

| generation | period of time | technology | characteristic |
|---|---|---|---|
| 1 | 1950 - 1959 | vacuum tube | mainframe |
| 2 | 1960 - 1968 | transistor | mainframe, process control computer |
| 3 | 1969 - 1977 | Integrated circuits | minicomputer |
| 4 | 1978 - | LSI, VLSI | workstation PC |
| 5 | 1993 - | LSI, VLSI | parallel processing |

Bild 1.1 Rechnergenerationen

Die Einteilung von Prozessoren in Generationen ist mehr historischer Natur. Eigenschaften moderner Prozessoren lassen sich mit dieser Klassifikation nur sehr schlecht beschreiben. So fällt der sehr unterschiedlichen Aufwand der in modernen Prozessoren bei der Implementation von parallel arbeitenden Komponenten betrieben wird nur in eine einzigen Klasse (Generation 5). Für die Charakterisieung eines Prozessors ist es aber wesentlich, ob er mit mehreren Prozessorkernen arbeitet (z.B. Pentium D) oder mit mehreren parallel arbeitenden Rechenwerken operiert (z.B. Rechenwerk für Integerzahlen und Rechenwerk für Floating-Point-Zahlen) oder nur eine interne Parallelverarbeitung von Befehlen kennt (z. B. Pipeline-Verarbeitung).

- Einteilung der Rechner im Hinblick auf zentrale oder verteilte Systeme

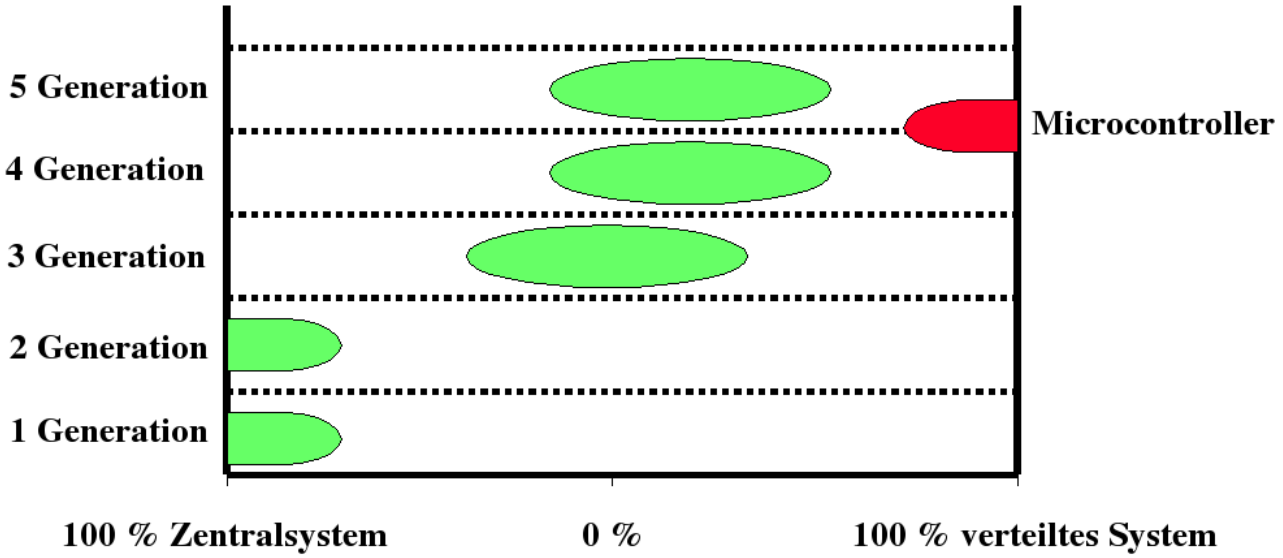



Bild 1.2 Zentrale und dezentrale Rechnergenerationen

- Übersicht einiger bedeutender Prozessoren

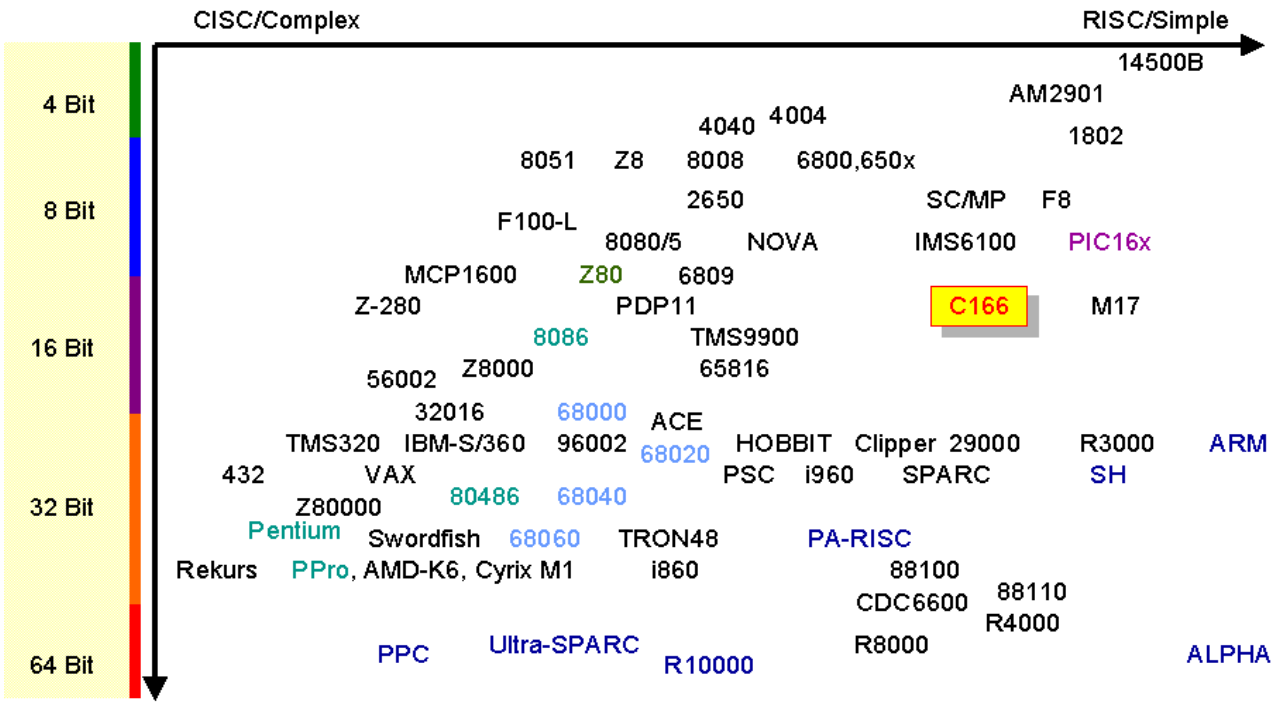



Bild 1.3 Prozessorfamilien

Die in Bild 1.3 dargestellten Prozessorfamilien sind nicht nur ihrer Wortbreite (meist die Wortbreite der Register oder die Wortbreite der prozessorinternern Bussysteme) nach sortiert sondern auch entsprechend der zugrunde liegenden Architektur. Die Architektur spiegelt sich in dem Aufbau der Befehlssätze des Prozessors wieder. Hier unterscheidet man zwischen CISC- (complex instruction set computer) und RISC- (reduced instruction set computer)Architekturen. Mit den RISC-Architekturen wurde erstmalig eine auf Befehlsebene durchgeführte Parallelisierung implementiert.

## 1.2 Prozessorgattungen

|| **Definition 1.1** Prozessoren lassen sich zu **Prozessorgattungen** zusammenfassen.

*Kennzeichung einer Prozessorgattung vor dem Hintergrund von Applikationen*

- Mikroprozessoren
- Mikrocontroller
- Signalprozessoren

- stark unterschiedliche wirtschaftliche Bedeutung
- Wechsel zwischen einzelnen Prozessorgattung nur bedingt sinnvoll

Wie erwähnt werden Mikroprozessoren, Mikrocontroller und Signalprozessoren auch als Prozessorgattungen gekennzeichnet. Hinsichtlich der Anzahl der Anwendungen überwiegen die Mikroprozessoren bei weitem die beiden anderen Gattungen. Versucht man das Entwicklungspotential der drei Gattungen abzuschätzen, so sind bei den Mikroprozessoren meist evolutionäre Veränderungen zu verzeichnen. Bei den Mikrocontroller sind z.Z. die größten Anderungen zu beobachten. Signalprozessoren dagegen weisen ein zwar kleines Marktsegment auf, dieses Segment ist jedoch schon über lange Zeiten stabil.

### Mikroprozessoren

- besteht aus Zentraleinheit (CPU mit Steuer- und Rechenwerk)
- Schnittstellen zum Betrieb prozessorexterner Speicher und Ein-/Ausgabeeinheiten
- Entwicklungswerkzeuge (Compiler, Linker, Betriebssysteme) für die Prozessorgattungen sehr unterschiedlich
- funktionsfähiges Rechnersystem setzt folgende weitere Komponenten voraus:
  - Verwendung zusätzlicher Logikschaltungen (Chipsatz) zum Betrieb einer Arbeitsspeichers
  - Implementation von Schnittstellen (serielle und parallele Schnittstellen)
  - Verwendung eines Betriebssystems
  - Implementation grafischer Interfaces (GUI graphical user inferface)s
  - typische Anwendung -> PC oder Workstation

### Mikrocontroller

- besteht aus Zentraleinheit (CPU mit Steuer- und Rechenwerk) plus controllerinternen Speicher
- controllerinterne Speicher als ROM oder RAM-Speicher realisiert
- Schnittstellen zum Betrieb prozessorexterner Speicher und Ein-/Ausgabeeinheiten
- weitere Schnittstellen:
  - digitale Ein- und Ausgänge
  - meist analoge Eingänge
  - serielle Ein- und Ausgänge
  - Implementation von Busschnittstellen
  - Implementation von digitalen Zähleinheiten im Controller
  - typische Anwendung -> Steuerung eines mechatronischen Systems z.B. ABS-System

### Signalprozessoren

- ähnlicher Funktionsumfang wie Mikrocontroller, jedoch größere Rechenleistung
- Bereitstellung der Rechenleistung durch:
  - superscalarer Aufbau (Bereitstellung mehrerer prozessorinterner Rechenwerke)
  - Befehlsformat sieht Programmierung mehrere parallel operierende Rechenwerke vor
  - typische Anwendung -> Video- und Audio-Anwendungen mit großen Anforderungen an die Rechenleistung

## 1.3 Rechnerarchitektur

> **Definition 1.2** **Rechnerarchitektur** Amdahl, Brooks und Blaauw definieren 1967: 'Rechnerarchitektur definiert sich aus den Attributen und des Verhaltens eines Computers, wie dieser von einem Maschinensprachenprogrammierer gesehen wird. Diese Definition umfasst den Befehlssatz, die Befehlsformate, die OP-Codes, die Adressierungsarten und alle Register und Speicher, die direkt durch einen Maschinensprachenprogrammierer verändert werden können. Die Implementation ist durch den aktuellen Hardwareaufbau, das logische Design und die Organisation der Datenpfade einer bestimmten Ausführung der Architektur definiert.'

- zwei bedeutsame Architekturen etabliert
    - Von-Neumann-Architektur (Verwendung eines gemeinsamen Speichers für Daten und Programme)
    - Havard-Architektur (Verwendung getrennter Speicher für Daten und Programme)

Die beiden Architekturen sind extrem unterschiedlich häufig anzutreffen. Alle Mikroprozessoren und einen Teil der Mikrocontroller entsprechen der von-Neumann-Architektur. Die Havard-Architektur ist bei einigen Mikrocontrollern und insbesondere bei Signalprozessoren anzutreffen. Der Hauptvorteil der Harvard-Architektur im Vergleich zur von-Neumann-Architektur ist deren schnellerer Zugriff auf prozessorexterne Daten. Bei dieser Architektur können prinzipbedingt in einem Zyklus zwei Daten gleichzeitig gelesen werden. Dies kann bei der von-Neumann-Architektur nur hintereinander erfolgen, wodurch diese Architektur einen langsameren Speicherzugriff hat.

### Von-Neuman-Architektur

- das Operationprinzip eines von-Neumann-Rechners ist durch den Aufbau, das funktionale Verhalten und die Befehlsbearbeitung gekennzeichnet
- a) physikalischer Aufbau bestehend aus
    - Eingabeeinheit
    - Ausgabeeinheit
    - Rechenwerk (Operationswerk)
    - Steuerwerk
    - Speicher

- b) das funktionale Verhalten ist gekennzeichnet von den im Rechner (Speicher) abgelegten Informationstypen
    - Maschinenbefehle
    - Daten
    - Adressen

- c) Befehlsbearbeitung
    - die Befehlsverarbeitung erfolgt nach 2-Phasen-Prinzip (kann u.U. noch feiner aufgegliedert werden)
        - i) Befehlshol- und Interpretationsphase (jeweils 1 Befehl wird geladen)
        - ii) Befehlsausführphase (jeweils ein Ergebnis wird bestimmt) -> Prinzip: SISD (single instruction, single data)

- Zusammenwirkung von Prozessorarchitektur und Programmstruktur
    - durch Architektur ist lineare Programmstruktur vorgegeben
    - lineare Befehlsbearbeitung unterbrochen durch
        - Sprungbefehle
        - Unterprogramme
        - Interrupt-Service-Routinen

## 1.4 Entwurfskriterien der Rechnerarchitektur

### Gestaltungsgrundsätze für Rechnersysteme

- Leistungsfähigkeit
  - Bestimmung der Leistungsfähigkeit durch:
  - a) Benchmarkprogramme (spezielle Testprogramme)
  - b) Angabe von MIPS (million instructions per second) oder Angabe von MFLOPS (millon floating point operations per second)
  - Angabe der Bewertungen nur sinnvoll unter Berücksichtigung späterer Anwendungsgebiete

Obwohl die Leistungsfähigkeit ein so offensichtliches Kriterium für die Entwicklung eines neuen Rechnersystems ist, ist sie schwer vergleichbar zu ermitteln. Dies drückt sich insbesondere bei den Maßzahlen MIPS und MFLOPS aus. In diesen Zahlen ist die Taktfrequenz dividiert durch Maßzahlen für die Dauer einer Befehlsverarbeitung verschlüsselt. Werden nun Prozessoren miteinander verglichen, die stark unterschiedlich große Befehlssätze haben, so wird derjenige Prozessor mit dem größeren Befehlssatz prinzipiell schlechter abschneiden als derjenige mit dem kleineren Befehlssatz.

Benchmarkprogramme zeigen diese Problematik nicht so ausgeprägt. Die Ergebnisse, die mittels Benschmarkprogrammen erzielt worden sind, sind leider auch nicht direkt auf eine spezielle Anwendung übertragbar, weil in derartigen Programmen auch nur ein Teil möglicher Befehle benutzt werden.

- Ausfalltoleranz
  - Eigenschaft von Systemen, bei Ausfall von Komponenten einen Minimal-Betrieb zu garantieren

- Konsistenz
  - Eigenschaft zum folgerichtigen Aufbau; z.B. Befehle zur Adressierung gleich, unabhängig von der Rechenoperation

- Orthogonalität
  - funktionell unabhängige Teilsysteme spezifizieren
  - jedes Teilsystem kann komplette Funktionen anderer Teilsysteme verwenden, z.B. Verwendung der Funktionen der Adressierungseinheit für Integer-ALU und Floating point-ALU

- Symmetrie
  - mathematische symmetrische Ausdrücke sollen entsprechend unterstützen werden, z.B. keine Unterscheidung des Wertebereichs von positiven oder negativen Zahlen

- Transparenz
  - Funktionen sollen überschaubar sein –¿ Dokumentation des Ergebnisses der Funktion (Operation) als Statusmeldung)

- Virtualität
  - physikalische Begrenztheit verbergen, z.B. Betrieb eines virtuellen Speichersystems in einem Rechner mit einer MMU (memory managment unit)

- Kompatibilität
  - gleicher Arbeitsauftrag führt bei verschiedenen Rechnern, unabhängig von Zeit und Ausrüstung, zu gleichem Ergebnis -> Programmkompatibilität

- dynamische Erweiterbarkeit
  - Erweiterung des Umfangs des Maschinenbefehlssatzes, z.B. durch die Mikroprogrammierung des Steuerwerkes des Mikroprozessors

Die oben aufgeführten Gestaltungsgrundsätze sollten nicht so verstanden werden, dass sie als Optimierungskriterien für den Entwurf eines neuen Rechners alle gleich gut zum Einsatz kommen könnten. Einige Grundsätze schließen sich gegenseitig aus (z. B. die Orthogonalität und die Kompatibilität) andere Grundsätze ergänzen sich sehr gut (z. B. die Orthogonalität und die Symmetrie). Bei der Konzeption eines neuen Rechners ist also immer nur eine Teilmenge der Gestaltungsgrundsätze verwirklichbar.

## 1.5 Darstellung von Zeichen und Zahlen

**Kodierungen**

- Kriterien für geeignete Kodierungen
  - einfache technische Realisierungen
  - schnelle Kodierungen und Dekodierungen
  - Kodierungen von:
    - Buchstaben, Ziffern, Sonderzeichen
    - diskretisierten Signalen (Spannungen, Ströme, Video- / Audio-Signalen)
    - Verwendung von n Bit-Codes

- Kodierungen basieren auf Zahlensysteme
  - einige Zahlensysteme
  - Zeichenvorrat = Basis
  - Darstellung der Zeichen in unterschiedlichen Codes

In Zusammenhang mit der Kodierung ist die Kompression (Erzeugung möglichst kleiner Datenmengen) von großer Bedeutung. Bei der Kompression werden unterschiedliche statistische Eigenschaften der zu kodierenden Daten (Zeichen oder Signale) ausgenutzt, um geringe Dateigrößen zu erzielen. Bedeutsame Kodierungen sind z.B. die Huffman-Kodierung (Komprimierung von Bildern), die Tabellenkodierung (Komprimierung von Daten, im Programm WINZIP) und die arithmetische Kodierung

| | | |
|---|---|---|
| Dualsystem | Repräsentation im Rechner | Basis 2 |
| Quinärsystem | Zählen mit einer Hand | Basis 5 |
| Oktalsystem | Menschenlesbare Darstellung von Maschinenzahlen | Basis 8 |
| Dezimalsystem | Zählen mit den Fingern | Basis 10 |
| Doudezimalsystem | „Dutzend“, historisch bedeutsam | Basis 12 |
| Hexadezimalsystem | Menschenlesbare Darstellung von Maschinenzahlen | Basis 16 |
| Vigesimalsystem | Zählen mit Fingern und Zehen | Basis 20 |
| Sexagesimalsystem | Astronomie, Mathematik | Basis 60 |

Bild 1.4 Zahlensysteme

## Darstellung von Zeichen

- Verwendung des ASCII-Zeichensatzes
- für alphanumerische Ausgaben (z. B. Kommandozeilen)

- Kodierung eines Zeichens in einem Byte -> max 256 Zeichen möglich
  - neben der dezimaler Notierung auch hexadezimale Notierung gebräuchlich (00 - 0FFH)
  - erste 128 Zeichen (7 Bit-Kodierung) -> Buchstaben des Alphabets plus Sonderzeichen (00 - 07FH)
  - nächste 128 Zeichen (8 Bit Kodierung) -> weitere Sonderzeichen (IBM-Grafikzeichen) (80H - 0FFH)

| y \ x | 000 | 001 | 010 | 011 | 100 | 101 | 110 | 111 |
|---|---|---|---|---|---|---|---|---|
| 0000 | NUL 0 | DLE 16 | 32 | 0 48 | @ 64 | P 80 | ` 96 | p 112 |
| 0001 | SOH 1 | DC1 17 | ! 33 | 1 49 | A 65 | Q 81 | a 97 | q 113 |
| 0010 | STX 2 | DC2 18 | " 34 | 2 50 | B 66 | R 82 | b 98 | r 114 |
| 0011 | ETX 3 | DC3 19 | # 35 | 3 51 | C 67 | S 83 | c 99 | s 115 |
| 0100 | EOT 4 | DC4 20 | $ 36 | 4 52 | D 68 | T 84 | d 100 | t 116 |
| 0101 | ENQ 5 | NAK 21 | % 37 | 5 53 | E 69 | U 85 | e 101 | u 117 |
| 0110 | ACK 6 | SYN 22 | & 38 | 6 54 | F 70 | V 86 | f 102 | v 118 |
| 0111 | BEL 7 | ETB 23 | ´ 39 | 7 55 | G 71 | W 87 | g 103 | w 119 |
| 1000 | BS 8 | CAN 24 | ( 40 | 8 56 | H 72 | X 88 | h 104 | x 120 |
| 1001 | HT 9 | EM 25 | ) 41 | 9 57 | I 73 | Y 89 | i 105 | y 121 |
| 1010 | LF 10 | SUB 26 | * 42 | : 58 | J 74 | Z 90 | j 106 | z 122 |
| 1011 | VT 11 | ESC 27 | + 43 | ; 59 | K 75 | [ 91 | k 107 | { 123 |
| 1100 | FF 12 | FS 28 | , 44 | < 60 | L 76 | \ 92 | l 108 | \| 124 |
| 1101 | CR 13 | QS 29 | - 45 | = 61 | M 77 | ] 93 | m 109 | } 125 |
| 1110 | SO 14 | RS 30 | . 46 | > 62 | N 78 | ^ 94 | n 110 | — 126 |
| 1111 | SI 15 | US 31 | / 47 | ? 63 | O 79 | _ 95 | o 111 | DEL 127 |

|— Steuerzeichen —|— Schriftzeichen —|

Bild 1.5 ASCII-Kode

- weiterer Zeichensatz -> Unicode (Kodierung in einem 16 Bit Wort, exakt: max 21Bit mit 16 Bit Offset)

- Kodierung eines Zeichens in einem Byte -> max 256 Zeichen möglich
  - Kodierung verschiedenster Zeichensätze
  - Beispiele:
    - U + 0 - U + 007FH -> Basis Latin (entspricht ASCII-Kodierung)
    - U + 0370H - U + 03FFH -> Griechisch

## Darstellung von Zahlen

- quasi 'natürliche' Zahlen im Microcomputer sind Dualzahlen/Binärzahlen
  - Wertebereich (Basis) 2, Werte 0, 1
  - Anwendung: Schnittstellen, Implementation von Logiken
  - Definition in Hochsprachen -> Boolean

- Zusammenfassung von 4 Binärzahlen -> hexadezimale Kodierung
  - Wertebereich (Basis) 16, Werte 0, 1, .. 8, 9, A, B, C, D, E, F
  - Kennzeichnung:
    - folgendes 'H' z.B. 3AH (Intel Assembler)
    - führendes '$' z.B. $3B (Motorola Assembler)
    - führendes '0x' z.B. 0x3C (Hochsprache C)

- Kodierung von Dezimalzahlen
  - Wertebereich (Basis) 10, Werte 0, 1, .. 8, 9
  - im Microcomputerbereich nicht immer direkt implementiert
  - besondere Maschinenbefehle notwendig z.B. ABCD (MC68000)
  - wenn nicht direkt implementiert:
    - Verwendung des Standardbefehlssatzes (basierend auf hexadezimale Notierung) und anschließende Korrektur des Ergebnisses (z. B. in der 8086-Architektur)
    - BCD-Zahlen können in ASCII-Kodierung vorliegen
    - 0, 1, .. 8, 9, -> 30H, 31H, .. 38H, 39H
    - typische Anwendung: numerische Ein-/Ausgaben, Finanzbuchhaltung

## vorzeichenlose Zahlen

- Wertebereich bedingt durch Anzahl Bits

  - 8 Bit -> 0 .. 255 (0 .. 0FFH)
  - 16 Bit -> 0 .. 65535 (0 .. 0FFFFH)
  - 32 Bit -> 0 .. 4294967295 (0 .. 0FFFFFFFFH)

- Verwendung:
  - Zeiger in Hochsprache C
  - Adressen und Adressrechenoperationen in Assembler
  - Programmierung von Schnittstellen in Assembler

## vorzeichenbehaftete Zahlen

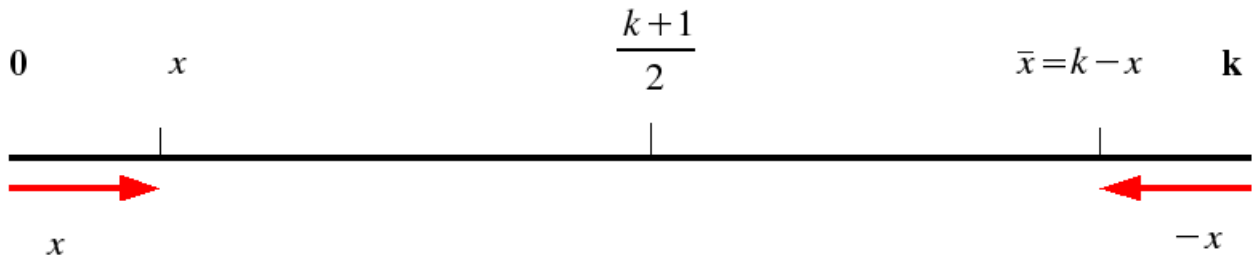



Bild 1.6 Zahlengerade bis zum Wert k

$$Definition\,des\,Komplements\,x\,zur\,Zahl\,k \;\rightarrow\; \overline{x} = k - x$$

$$Differenz\,D = y - x = y + \overline{x} - k = y + k - x - k$$

$$positiver\,Wertebereich\,[0 - \frac{k+1}{2}]$$
$$negativer\,Wertebereich\,[\frac{k+1}{2} - k]$$

- höchstwertiges Bit für Vorzeichen reserviert
  - Bit gesetzt (Bit = 1) -> Zahl negativ
  - 1Bit gelöscht (Bit = 0) -> Zahl positiv

- Darstellung des Komplements
  - Einerkomplement -> Invertieren aller Bitstellen

**Beispiel 1.1**

$$Hexadezimalzahl\ \ x = 0AH \rightarrow Binärzahl\ \ (Byte) \rightarrow 00001010 =$$
$$0 \cdot 2^7 + 0 \cdot 2^6 + 0 \cdot 2^5 + 0 \cdot 2^4 + 1 \cdot 2^3 + 0 \cdot 2^2 + 1 \cdot 2^1 + 0 \cdot 2^0$$
$$Hexadezimalzahl\ \ \overline{x} = 0F5H \rightarrow Binärzahl\ \ (Byte) \rightarrow 11110101 =$$
$$1 \cdot 2^7 + 1 \cdot 2^6 + 1 \cdot 2^5 + 1 \cdot 2^4 + 0 \cdot 2^3 + 1 \cdot 2^2 + 0 \cdot 2^1 + 1 \cdot 2^0$$

$$\begin{aligned} Darstellung\ \ von\ \ 0AH &= +10_{Dez} \\ 0F5H &= -10_{Dez}\ \ ist\ \ eindeutig \end{aligned}$$

- Darstellung der Zahl 0 bereitet Probleme

$$x = +0 \rightarrow Binärdarstellung(Byte)\,00000000$$
$$x = -0 \rightarrow Binärdarstellung(Byte)\,11111111$$

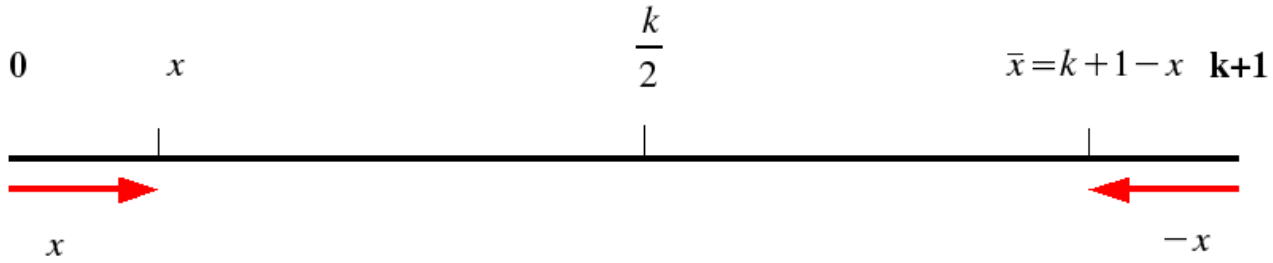



Bild 1.7 Bildung des Zweier-Komplements zum Wert k

- aus Einerkomplement -> Zweierkomplement
- Bildung des Zweierkomplements
  - Invertierung der Binärzahl und Addition +1

- Wertebereich bedingt durch Anzahl Bits
  - 8 Bit -> -128 .. 0 .. +127 (80H .. 0 .. 7FH)
  - 16 Bit -> -32768 .. 0 .. +32767 (8000H .. 0 .. 7FFFH)
  - 32 Bit -> -2147483648 .. 0 .. +2147483647 (80000000H .. 0 .. 7FFFFFFFH)

## gebrochen-rationale Zahlen

- zusätzliche Darstellung des Wertebereichs für 0 <= x < 1
- gebrochen-rationale Zahlen werden immer vorzeichenbehaftet dargestellt
- Darstellung als:
  - Floating-Point-Zahl
  - Fix-Point-Zahl

- Floating-Point-Zahl (Definition in C als 'float' oder 'double')
  - Zahl ist grundsätzlich normiert entsprechend:

$$Zahl = Vorzeichen \cdot 1.Mantisse \cdot 2^{Exponent}$$

  - Vorzeichen -> Gesamtvorzeichen der Zahl (1 Bit)
  - Mantisse -> grundsätzlich positiv (23 Bit float, 52 Bit double)
  - Exponent -> vorzeichenbehaftet (8 Bit float, 11 Bit double)

- Problem: die Zahl 0 ist nicht darstellbar
- Lösung: Verwendung einer Kombination aus dem Exponenten (negative Zahlen im Exponenten haben einen Wert mehr!)

**Beispiel 1.2** Float-Zahlen

$$Exponent - 128 \longleftrightarrow 0 \longleftrightarrow +127 \ (-128 \ ist \ überzählige \ Kombination)$$

$$Addition \ Offset \ 127$$

$$Exponent(-127 \longleftrightarrow 0 \longleftrightarrow +127) \rightarrow (0 \longleftrightarrow 127 \longleftrightarrow 254)$$

$$Kombination \ 255 \ frei \ und \ reserviert$$

- Exponent 0FFH (float) plus Mantisse zur Kodierung von Sonderzeichen (z.B. 0, -unendlich +unendlich)

- Fix-Point-Zahl
- Komma an fester Position (meist zwischen zwei Byte oder zwei 16 Bit Worten)
- eine Normierung findet nicht statt
- der ganzahlige Teil (Integer) enthält das Vorzeichen, die Mantisse ist immer positiv

$$\begin{aligned} Zahl &= Integer.Mantisse \\ Zahl &= 2^{n-1} \ldots 2^0.2^{-1} \ldots 2^{-n} \end{aligned}$$

- Besonderheiten:
  - die Zahl 0 ist unmittelbar darstellbar
  - Multiplikationen bedeuten einer Verdoppelung der Wortbreite
  - Fix-Point-Zahlen können mit Standard-Maschinenbefehlssatz bearbeitet werden (kein FPU erforderlich)

# Kapitel 2

# Prozessorinterne Speicher

## 2.1 Elektronische Speicher

Der Arbeitsspeicher eines Rechners stellt ein wichtiges Charakteristikum für die Leistungsfähigkeit eines Rechners dar. Hier sind nicht nur die Größe des Arbeitsspeicher und die Geschwindigkeit des Speicherzugriffs zu benennen. Die bedeutsame Stellung des Speichers in einem Rechner ergibt sich auch aus der zugrunde liegenden Architektur, die bei den meisten Systemen der von-Neumann-Architektur entspricht. Bedingt durch diese Architektur muß jede Aktion des Prozessors zeitlich aufeinander folgend mittels des Speichers erfolgen. Dies bedeuten, das nacheinander ein Befehl aus dem Speicher gelesen, die zugehörigen Daten aus dem Speicher gelesen und schlussendlichen die gewonnenen Ergebnissen wiederum im Speicher angelegt werden müssen. All diese Zugriffe auf den Speicher können nur zeitlich aufeinander erfolgen. Einen parallelen Zugriff sieht die von-Neumann-Architektur nicht vor.

- der Arbeitsspeicher stellt in modernen Rechnersystemen eine Grenze für die Steigerung der Leistungsfähigkeit dar
- Charakterisierung nach Kapazität und Zugriffzeit

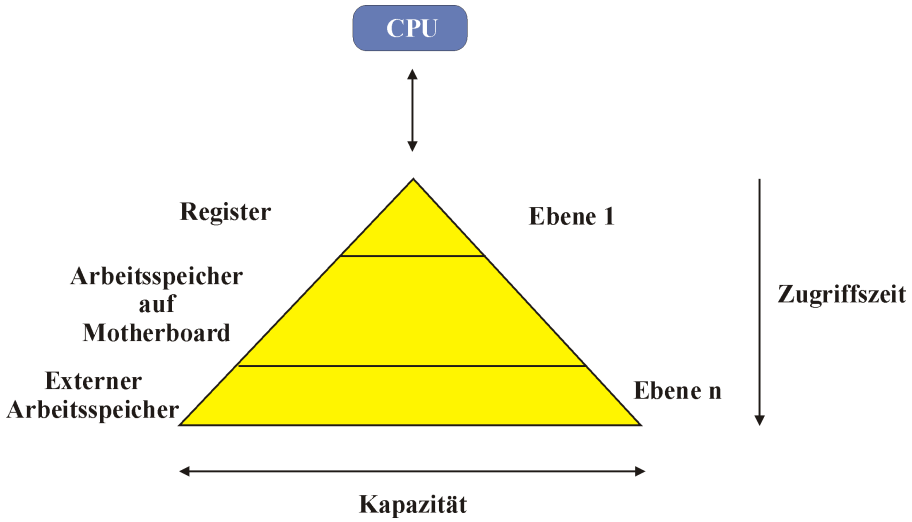



Bild 2.1 Speicher in einem Rechner

| Art der Speicher | Zugriffzeit | Kapazität |
|---|---|---|
| Register | 1ns | 100 |
| Cache | 2 . . . 10ns | 16 . . . 512 kByte |
| Hauptspeicher | 70ns | 256 . . . 4096 MByte |
| Festplatte | 5ms | 120 GByte |

- Elektronischer Speicher eines Rechners lässt sich einteilen in:
  - Register
  - Statischer Speicher (SRAM)
  - Dynamischer Speicher (DRAM)
  - Festwertspeicher (ROM)

### Register

- kürzeste Zugriffszeiten
- direkt adressierbar (im Maschinenbefehl)
  - von Prozessoren muss kein spezielle Adressierungseinheit bereitgestellt werden
- verfügbar als Adressregister, Datenregister, Akkumulatoren
- verwendet als Einzelregister
  - z. B. EAX, EBX, ESI (Pentium)
- verwendet als Registerfile
  - EMM0 - EMM7 (Datenregister für SSE-Pentium)

### Statischer Speicher (SRAM)

- kurze Zugriffszeiten
- als prozessorinterner oder prozessorexterner Speicher realisiert

  - prozessorinterner Speicher
    - L1-Cachespeicher (nicht direkt adressierbar)
    - Internes RAM (bei Signalprozessoren oder Controllern)
    - -> im Adressraum des Prozessors

  - prozessorexterner Speicher
    - L2-Cachespeicher (nicht direkt adressierbar)
    - im Standardarbeitsspeicher eines Prozessors (insbesondere bei Signalprozessoren oder Controllern)

- statische Speicher brauchen wenig Overhead für den Betrieb
- relativ teuer, relativ hohe Verlustleistungen
- relativ kleine Speichergrößen

Der statische Speicher (SRAM) erfordert den geringsten Aufwand hinsichtlich der Implementation von externen Speichersystemen. Lediglich eine Adresskodierung und Treiberschaltungen sind vorzusehen. Nachteilig bei statischen Speichern ist deren große Verlustleistung im Vergleich mit den dynamischen Speichern.
Bei den internen statischen Speichern (prozessorinterner RAM-Speicher oder L1-Cache-Speicher) ist die potetielle Nutzung des Prozessors zu beachten. Wird ein Prozessor als 'General Purpose-Prozessor' z. B. in einem PC eingesetzt, so findet man nur einen Cache-Speicher jedoch kein prozessorinternes RAM. Der Grund besteht darin, dass bei derartigen Prozessoren von mehreren quasi parallel arbeitenden Tasks ausgegangen werden muss. (Verwendung der Betriebssysteme

WINDOWS oder UNIX). Bei jedem Taskwechsel müsste dann der komplette interne RAM-Speicher auf dem Stack gerettet (entsprechend großer Stackframe) und bei der Rückkehr zu der unterbrochenen Task wieder restauriert werden. Der Gewinn durch den schnellen Zugriff auf dem internen RAM-Speicher ginge somit verloren.
Anders verhält sich einer Anwendung bei der ein Mikrocontroller oder ein Signalprozessor zum Einsatz kommt. Hier kann in den überwiegenden Fällen von lediglich einem Anwendungsprogramm ausgegangen werden. In diesem Fall können sehr häufig benutzte Variablen in dem prozessorinternen Ram abgelegt und dadurch ein schnellerer Speicherzugriff gewährleistet werden. Folgerichtig findet man in derartigen Prozessoren einen oder mehrere interne RAM-Speicher, jedoch keinen Cache-Speicher.

## Aufbau eines statischen Speichers (SRAM)

- jede Speicherzelle besteht aus einem Flipflop

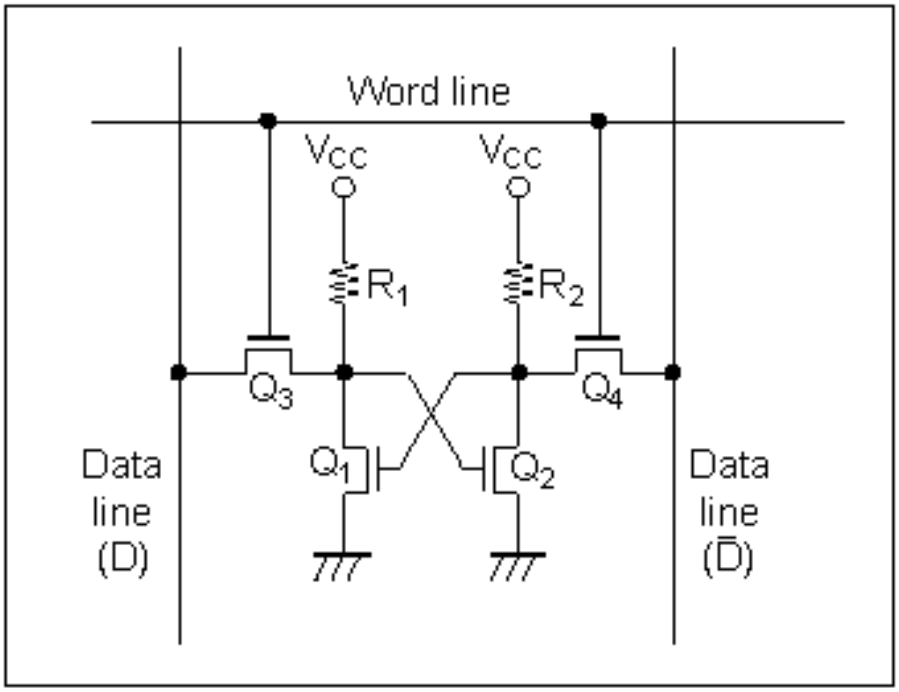



Bild 2.2 Speicherzelle eines SRAM

- bedingt durch die Verwendung eines Flipflops
  - wiederholtes Lesen möglich (keine Störung des gespeicherten Wertes durch Leseoperation)
  - der Speicherinhalt bleibt erhalten solange Versorgungsspannung bereitsteht
  - Lese- und Schreibgeschwindigkeit ist gleich

## Dynamischer Speicher (DRAM)

- Standardspeicher des Motherboards jedes Rechners

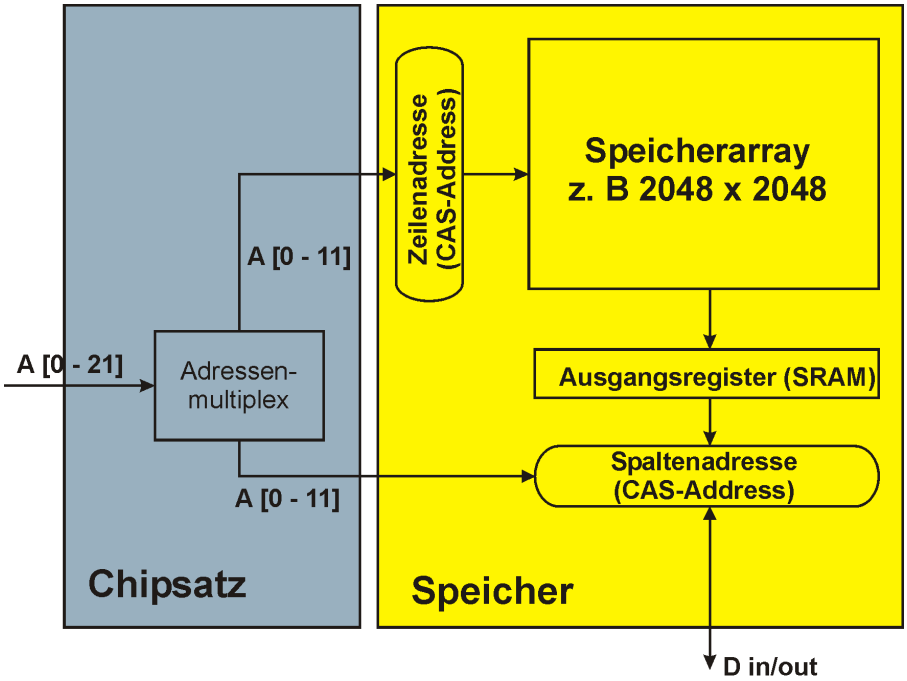



Bild 2.3 Prinzip des DRAM

- Kernzelle jeder Speicherzelle ist ein Kondensator
  - daher Verlust der gespeicherten Information nach bestimmter Zeit
  - Auslesen der Information ist zerstörend (Ladung des Kondensator gelöscht)
  - Zur Aufrechterhaltung der Ladung ist ein ?Refresh? notwendig

- längere Zugriffszeiten (prinzipiell)
  - Unterschiedliche Varianten von DRAM
  - EDO-DRAM (extended data out DRAM)
  - SDRAM (synchron DRAM, Burst Zugriff ohne Anlagen der einzelnen CAS-Adressen)
  - DDR-SDRAM (double data rate, Zugriff an beiden Taktflanken eines SDRAM)
  - VDRAM (Video-DRAM, Bereitstellung eines speziellen Ausgangsschieberegisters)

- Vorteile DRAM
  - große Speicherbereiche
  - geringe Verlustleistungen
- Nachteile DRAM
  - aufwendige Ansteuerung
  - Notwendigkeit für Refresh-Ansteuerung

**Festwertspeicher (ROM)**

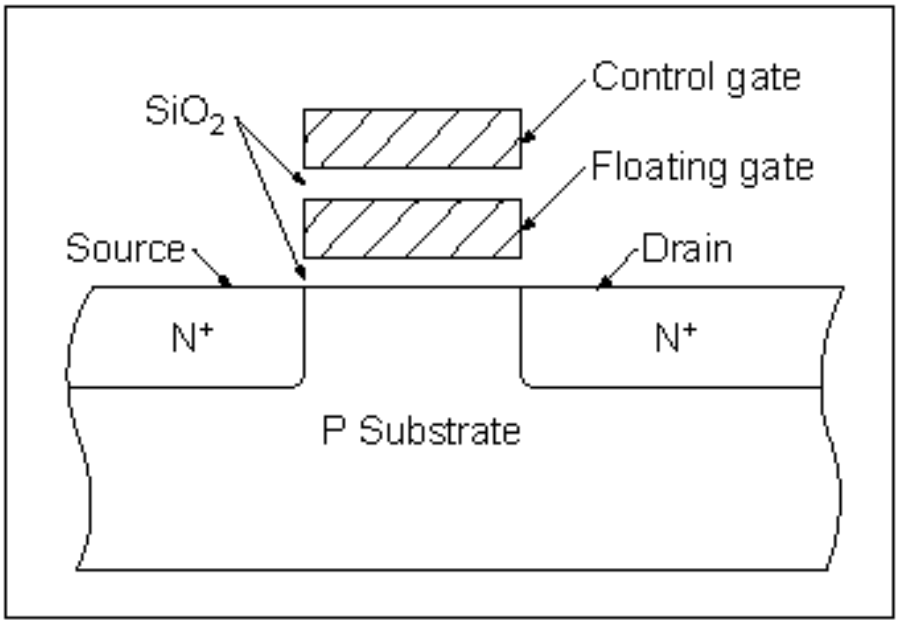



Bild 2.4 Speicherzelle eines EPROM-Speichers

- Kernzelle besteht aus einem FET-Transistor mit einem 'Floating Gate' (Gate hat keine Verbindung mit dem Source- oder Drain-Ein/Ausgang)
  - Ladung auf dem Gate bleibt daher auch bei Wegfall der Versorgungsspannung erhalten

- Verwendung von EPROM
  - Prozessorinterner Speicher bei Signalprozessoren oder Controller
  - Prozessorexterner Speicher z. B. BIOS des PC

- EPROM in unterschiedliche Varianten verfügbar
  - EPROM (elektrische schreibbar, löschbar mit UV-Licht)
  - EEPROM (elektrisch schreib- und löschbar)
  - FLASH-EPROM (elektrisch schreib- und löschbar in Speicherzeilen)

## 2.2 Adressierungsarten

- wesentliche Eigenschaft eines Prozessors, dokumentiert im Maschinenbefehlssatz
- Unterscheidung drei zwischen prinzipiellen Möglichkeiten
  - Linearer Adressierung
  - Segmentierte Adressierung
  - Virtuelle Adressierung

### Lineare Adressierung

- Standardverfahren bei einem von Neumann-Prozessor

- Zugriff auf dem gesamten Arbeitspeicher
  - zunächst keine Einschränkung der Zugriffsrechte
- Wortlänge des Adressregister entspricht dem externen Adressbus
  - Move (A0),D0 Lade Inhalt der Speicherstelle auf die A0 zeigt nach D0 (MC680000)
- bei Einsatz von Multiuser/Multitasking-Betriebsystemen ist relative Adressierung erforderlich

## Segmentierte Adressierung

- Einteilung des Arbeitsspeichers in Segmente
- Bereitstellung von Segmente für Teile eines Anwenderprogramms
  - Programm- (Code-,Text-)Segment
  - Daten-Segment
  - Stack (Heap)-Segment

- Adressierung innerhalb des Segmentes einfacher
  - Anwender, Compiler und Linker verfügen über Defaultwerte für Adressierung
  - Platzierung des Segmentes im Arbeitsspeicher ist Aufgabe des Betriebsystems
  - Verzicht auf relative und speicherindirekte Adressierungsverfahren

- Segmentierung ermöglicht Vergabe von Rechten für Speicherzugriff
- Verwaltung der Rechte ist Aufgabe des Betriebssystems
- Segmentierung kann tabellengestützt oder registergestützt erfolgen

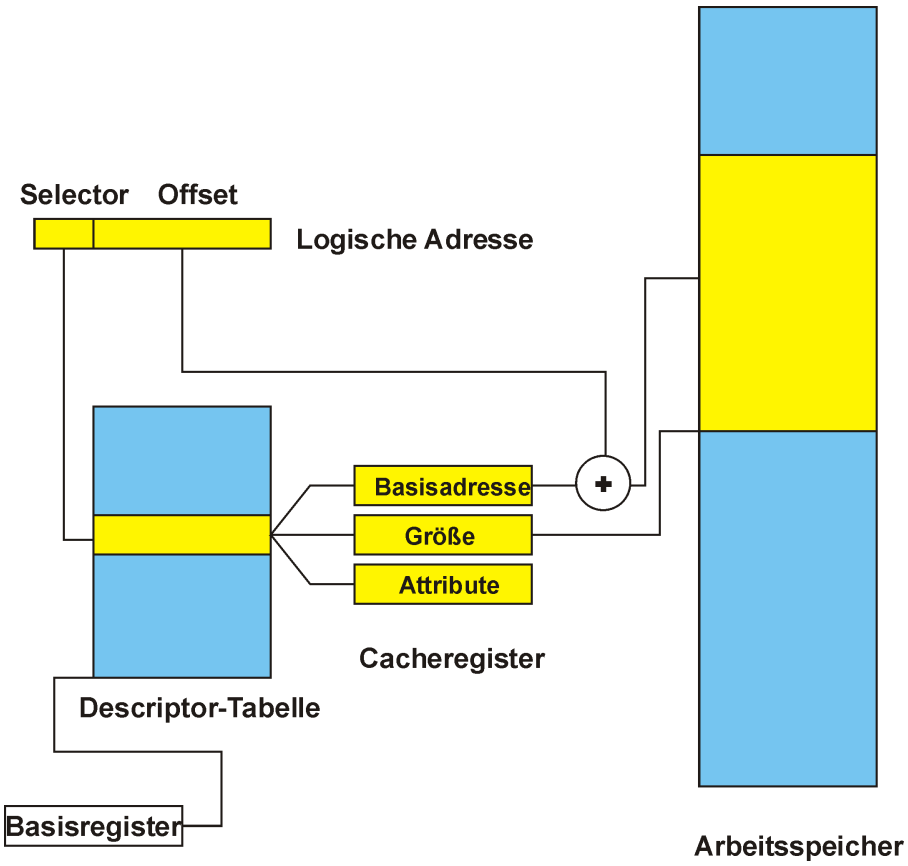



Bild 2.5 Tabellengestützte Segmentierung

## Virtuelle Adressierung

- notwendig für Multiuser/Multitasking-Betriebssysteme
  - Steuerung des Zugriffs auf den Speicher
  - Zerlegung des Speichers in einzelne Seiten (Pages)

- häufig einer Segmentierung nachgeschaltet
- Programmierung durch Betriebssystem

- virtuelle Adressierung erfolgt tabellengestützt
  - Verwendung von 1 oder 2 Tabellen
  - (page table und page directory)

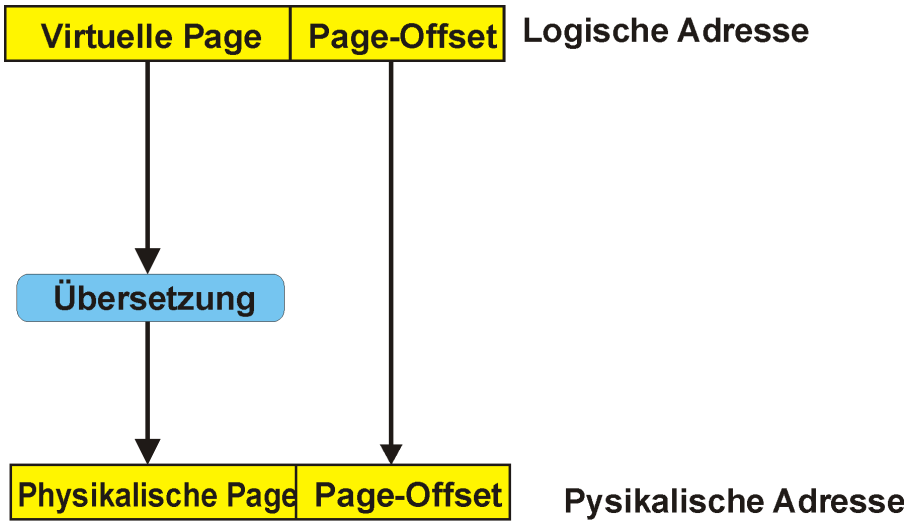



Bild 2.6 Virtueller Speicher

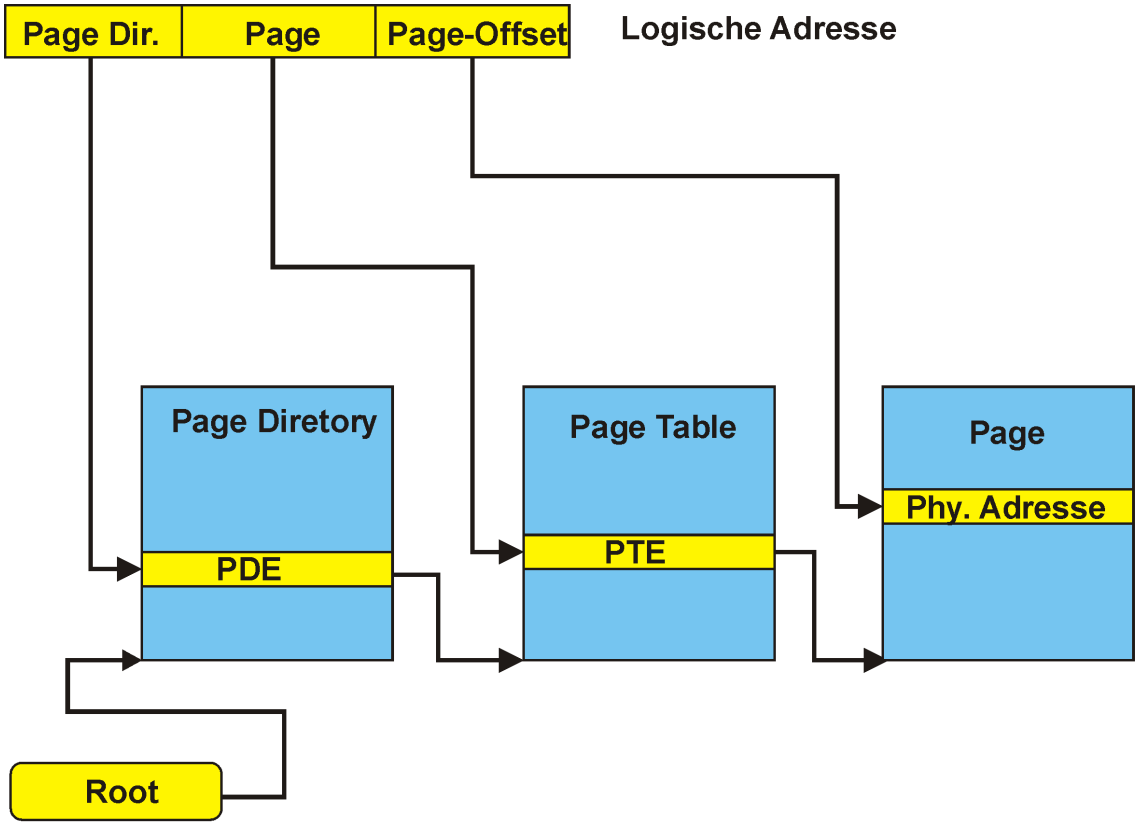



Bild 2.7 Zweistufige tabellengestützte virtuelle Speicheradressierung

## 2.3 Adressdekodierung

- Adressdekodierung ist eine logische Schaltung zur Auswahl bestimmter Adressbereiche

  - immer notwendig, wenn Adressbereich des Speichers kleiner ist als der des Prozessors
  - Verzicht auf Adressdekodierung bedeutet Spiegelungen des Speichers im Adressraum des Prozessors
  - Adressdecodierungen realisiert in:

    - Chipsatz (bei PC, Workstations)
    - FPGA, PLD (Signalprozessoren, Mikrocontroller)
    - interne Register (Mikrocontroller)

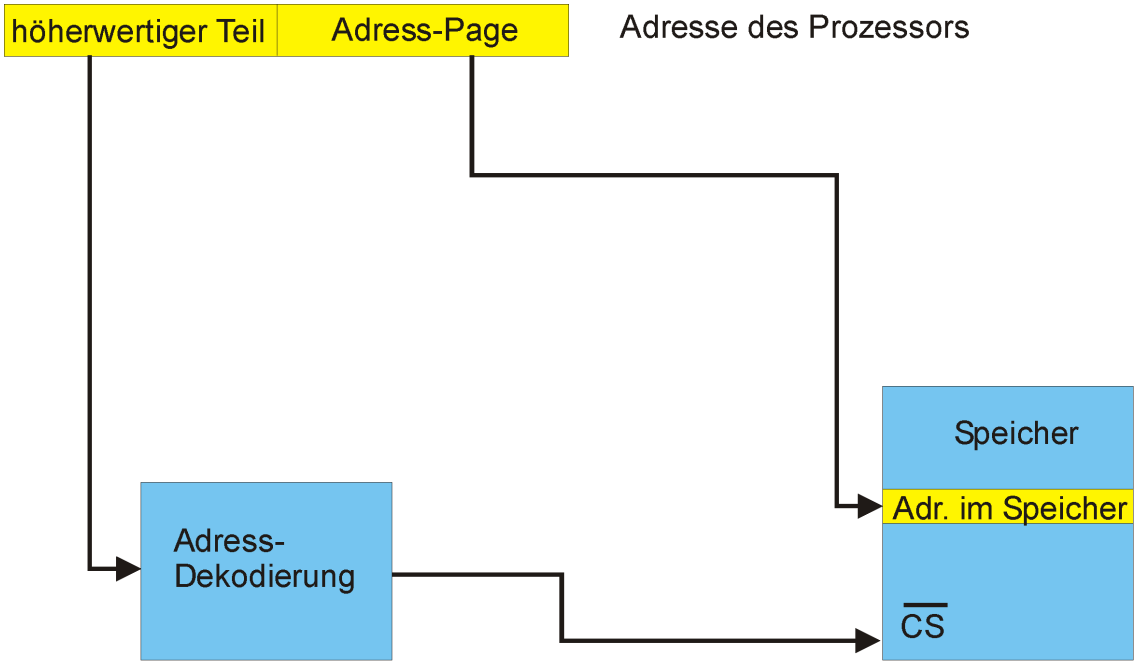



Bild 2.8 Adressdekodierung

# Kapitel 3

# Sekundäre Speicher

## 3.1 Sekundäre Speicher

Dieses Kapitel befasst sich mit dem Massenspeicher in einem Rechnersystem. Es werden die wichtigsten Eingenschaften von Magnetspeicher (Harddisc), deren prinzipielle Funktionsweise und deren Gebrauch als einzelne Einheit oder in einem Plattenstapel (RAID) vorgestellt. Des weiteren erfolgt eine Vorstellung von optische Speichermedien (CD und DVD).

- Sekundärspeicher setzt sich in modernen Rechner aus folgenden Komponenten zusammen
    - Festplattenspeicher (Harddisk)
    - CDROM / CDRAM
    - DVD-ROM / DVD-RAM

- Generelle Eigenschaften:
    - nicht flüchtiger Speicher (Speicherinhalt bleibt nach Lesezugriff oder Ausschalten der Versorgungsspannung erhalten)
    - Speicherzugriff erfolgt blockweise (minimale Blockgröße ≫1Byte, z.B. 2 oder 4 kByte)
    - Speicherzugriff unter Kontrolle des Betriebssystems
    - Dauer des Speicherzugriff sehr viel länger als Zugriff auf Arbeitsspeicher

## 3.2 Festplattenspeicher

- klassischer Sekundärspeicher in nahezu jedem Rechnersystem
    - Speicherung von Programmen
    - Speicherung von Daten
    - unterschiedlichste Bauformen und Speichergrößen

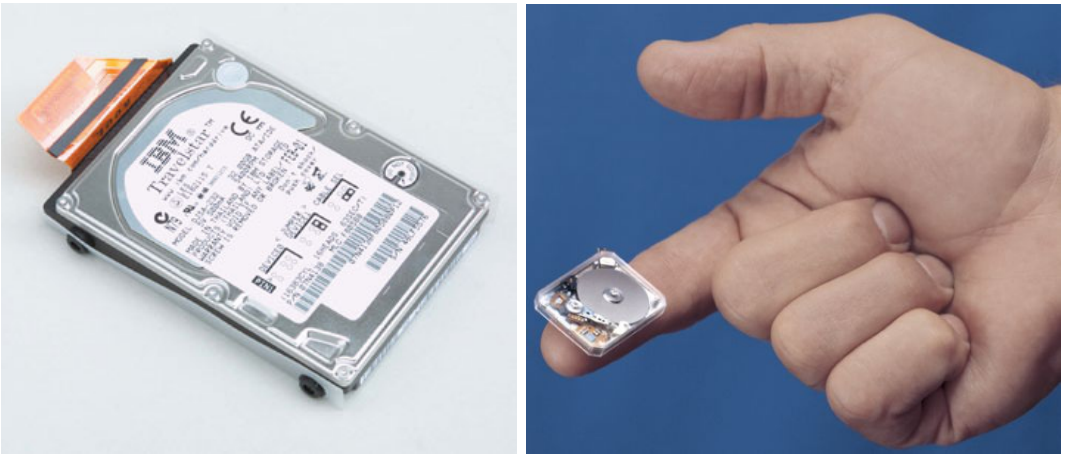


Bild 3.1 Harddisk in einem Recher bzw. einer Kamera

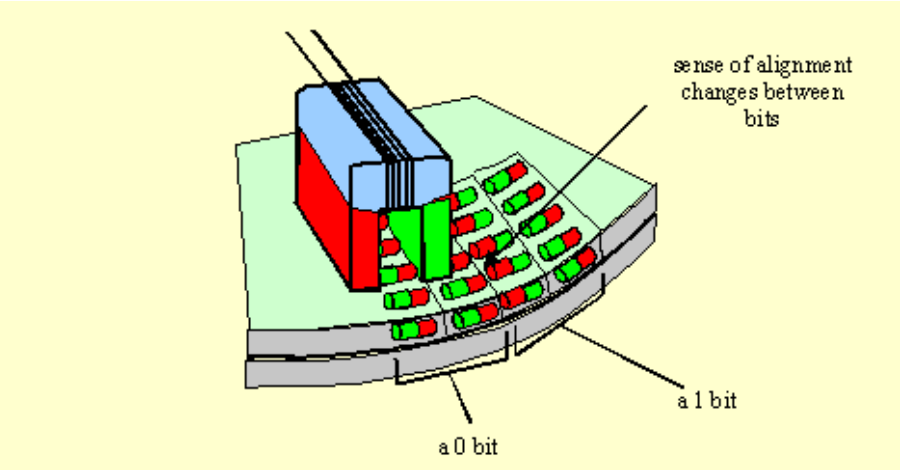



Bild 3.2 Prinzipielle magnetische Speicherung

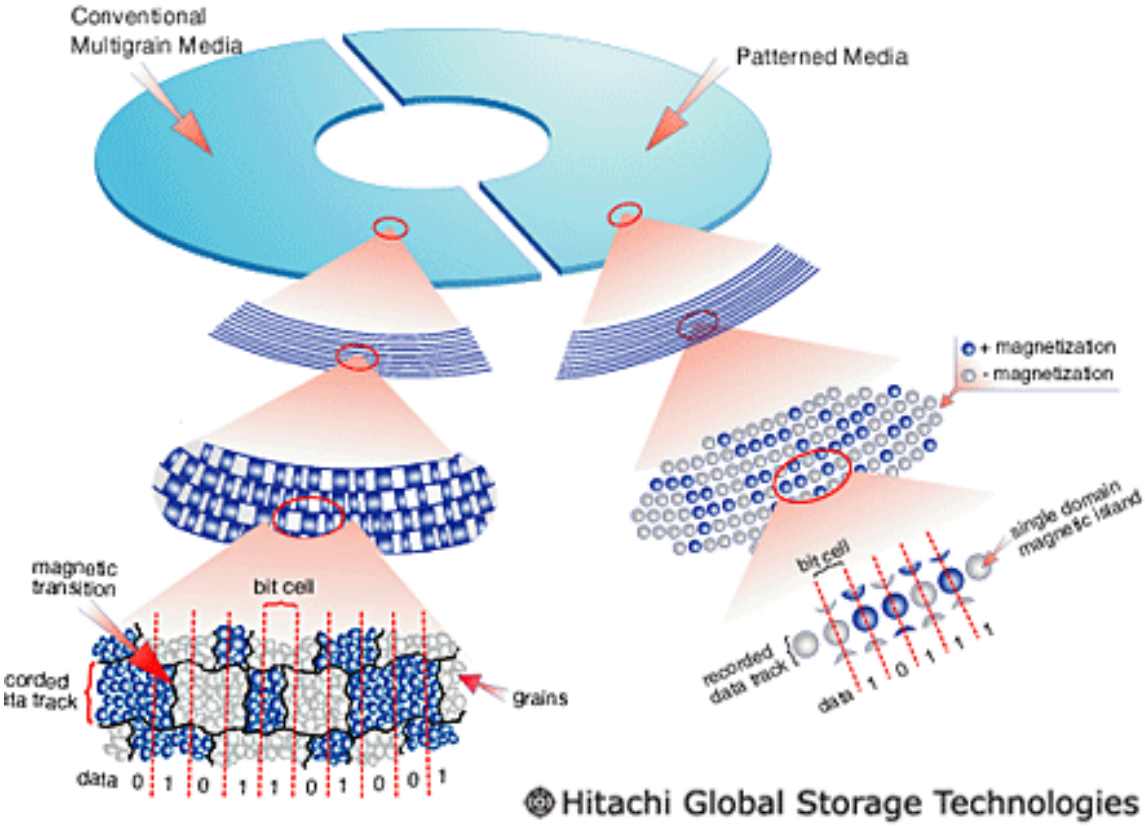


Bild 3.3 Verbesserte Struktur magnetischer Bezirke

- Aufbau einer Harddisk
  - Realisierung mit einem Plattenstapel
  - Einteilung einer Platte in Sektoren, Tracks und Zylinder

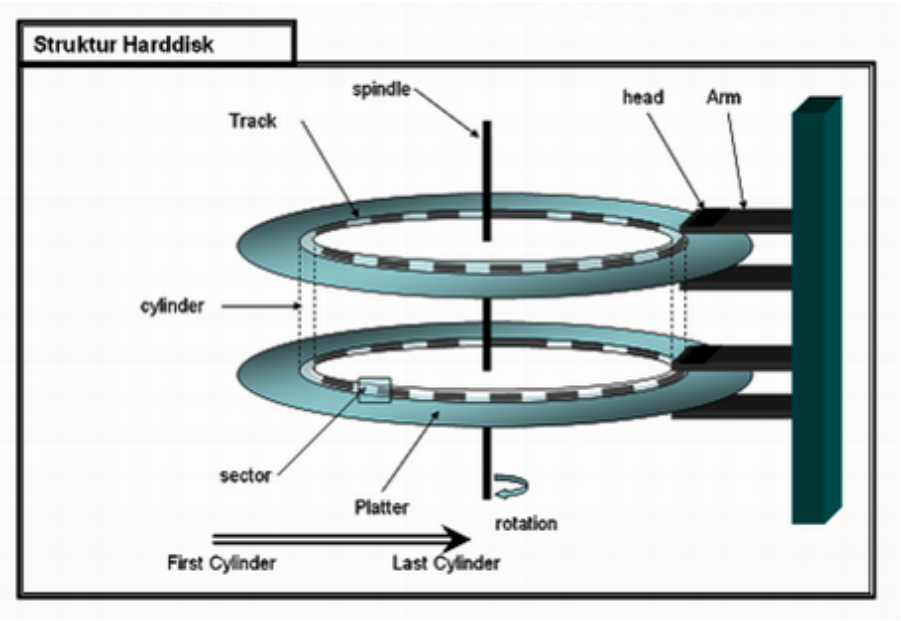



Bild 3.4 Aufbau einer Harddisk

- Magnetischer Flußwechsel zur Speicherung von 0 und 1
- Speicherung der Daten auf Harddisk im RRL-Code (run length limited)

**Beispiel 3.1**

$$0111\ \ 0001\ \ 1111\ \ 0000 \rightarrow RRL - Code = 1,3,3,5,4$$
$$max.\ Anzahl\ 0\ spezifiziert\ (d)$$
$$max.\ Anzahl\ 1\ spezifiziert\ (k)$$

- bevor Zugriff auf Harddisk ist eine Formatierung erforderlich
  - Formatierung bedeutet die Festlegung der Struktur der Datenspeicherung auf der Harddisk (Filesystem)
  - unterschiedliche Filesysteme gebräuchlich
    - FAT32 Windows (auch in anderen Betriebssystemen gebräuchlich)
    - NTFS Windows (von anderen Betriebssystemen meist nur Lesemöglichkeit)
    - Ext3, ReiserFS (Linux)
  - Formatierung bedeutet Verlust aller Daten auf der Harddisk!!
- Aufbau eines Filesystems
  - FAT (file allocation Table, ursprünglich für Betriebssystem DOS entwickelt)
  - verfügt über Blockstruktur

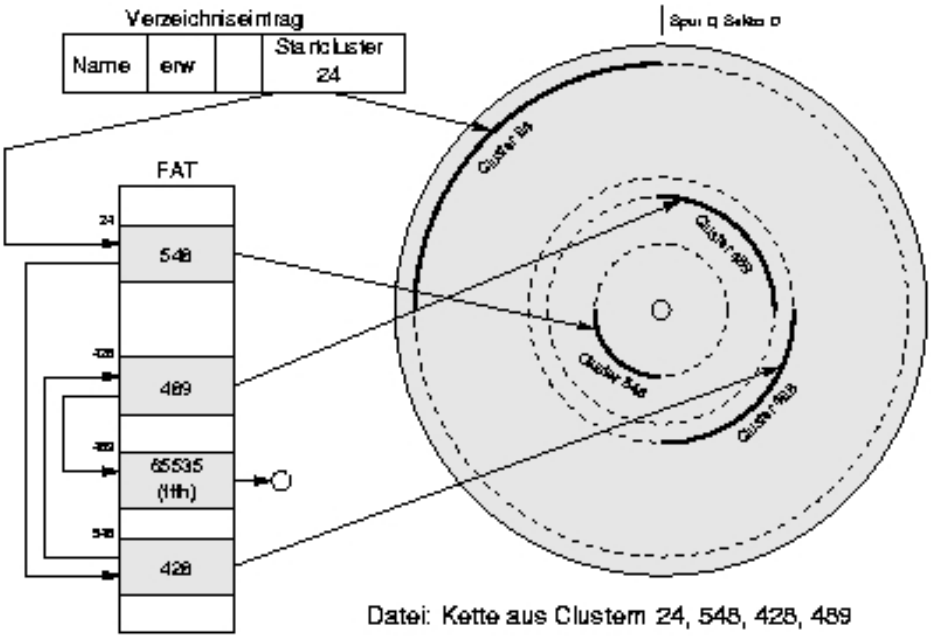



Bild 3.5 Speicherung von Daten im FAT-Filesystem

- Baumorientierte Filesystem (ext3, ReiserFS, NTFS)
  - Verwendung eines B-Baums (modifiziert)
  - Speicherung der Daten in 'Blätter' des Baums
  - Verzicht auf Blockstruktur
  - Einbau von 'Journaling'-Funktionen

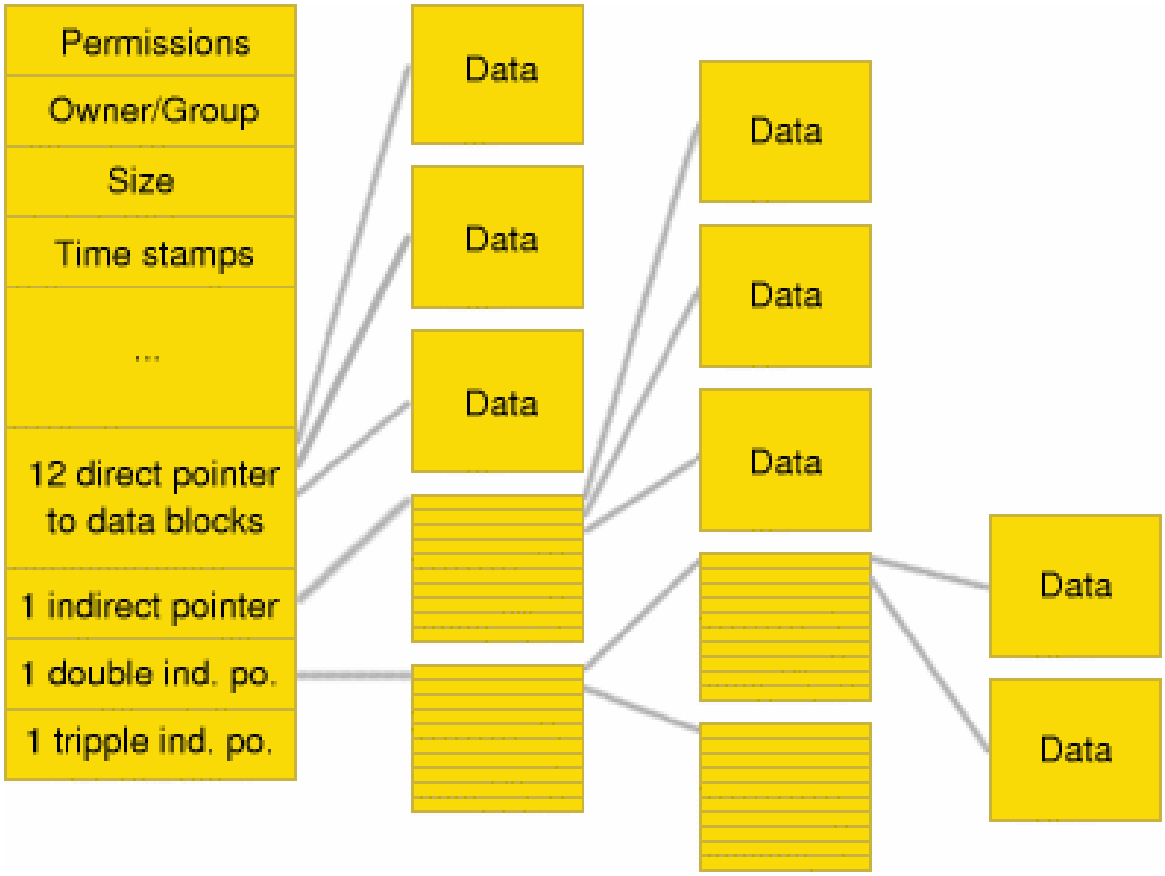



Bild 3.6 Baumorientiertes Filesystem (Inodes Linux)

- Verwendung mehrerer Harddisks in einem Rechner

  - RAID (redundant array of inexpensive disks)

    - Grundidee: Verwendung mehrerer 'kleiner' parallel operierender Disks statt einer 'großen' Disk
    - ursprünglich für Serverlösungen vorgesehen
    - heute in bestimmten Varianten auch im PC anzutreffen

    - durch parallel operierende Disks -> unterschiedliche Verhaltensweisen implementierbar

      - Verkürzung der Zeit für den Datenzugriff
      - Vergrößerung der Speicherkapazität
      - Bereitstellung von Redundanz bei der Datenspeicherung (mirroring, Fehlercodes)
      - RAID-Disks in verschiedenen Levels spezifiziert

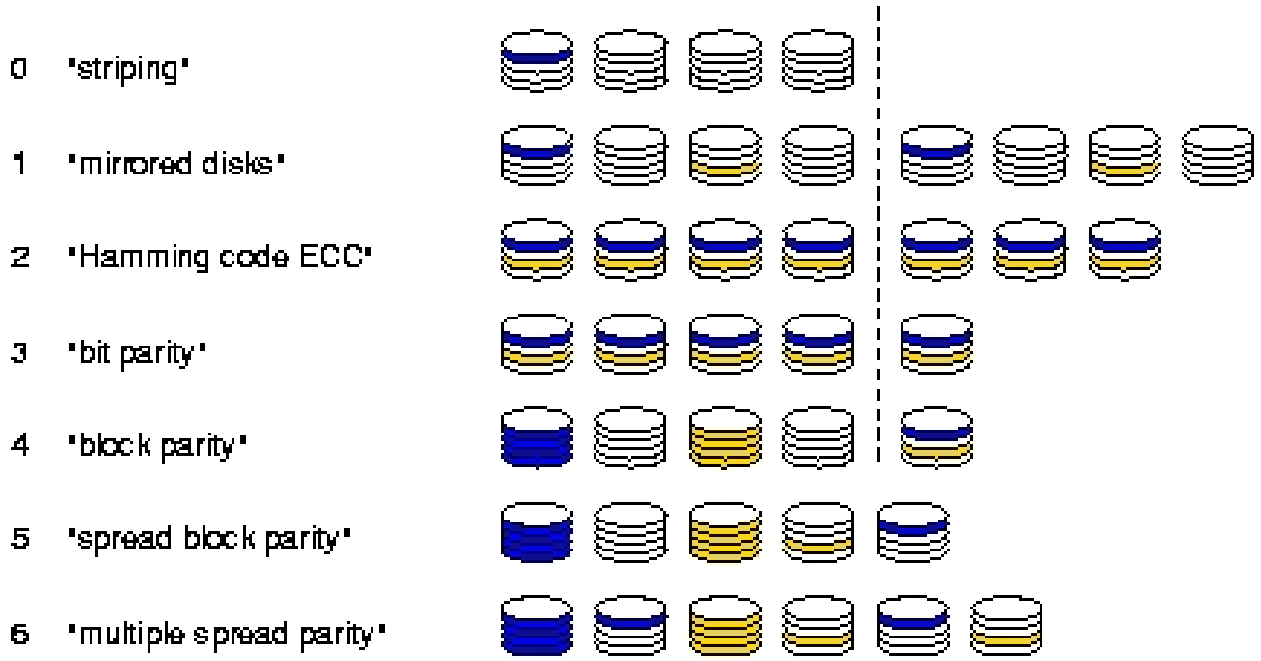



Bild 3.7 Verschiedene RAID-Level

## 3.3 CD/DVD

- neben Magnetbändern klassischer Massenspeicher

  - Charakterisierung als Massenspeicher heute kaum gerechtfertigt -> Harddisc verfügt über 20 - 40 fache Kapazität
  - optische Speichermedium mit wahlfreien Zugriff -> Zugriff auf Datenblöcke

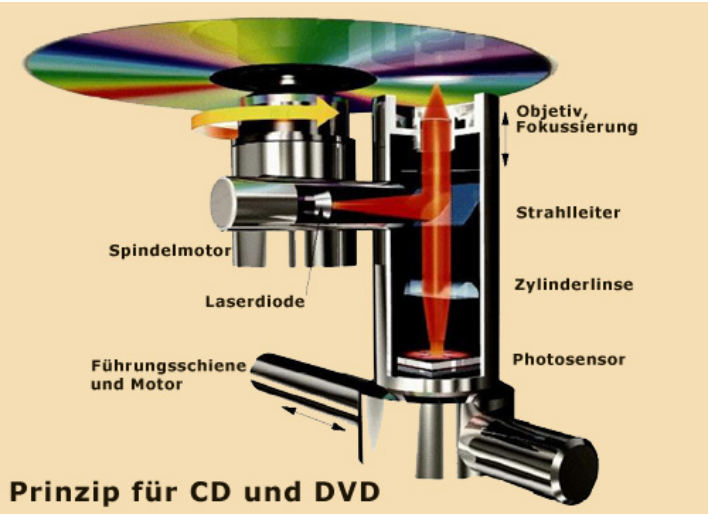



Bild 3.8 Prinzipieller Aufbau

- Funktionsweise der optischen Speicherung

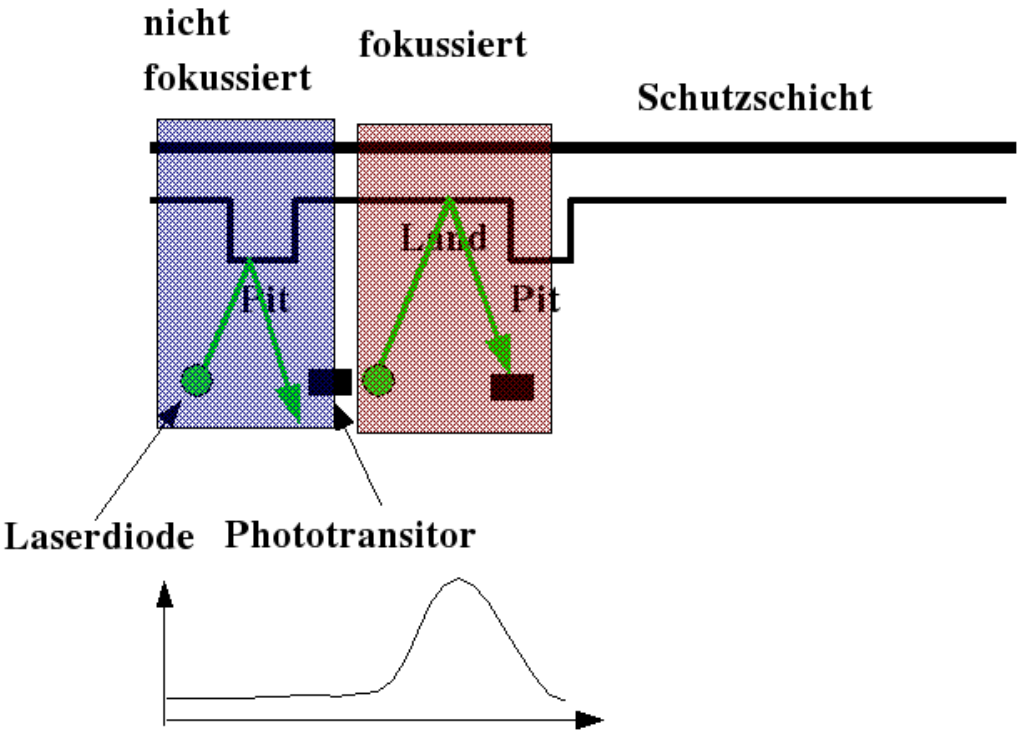



Bild 3.9 Prinzipielle Funktionsweise

- Urvater aller CD, DVD ist die Audio-CD (CD-A)

- Definition der CD in 'books'

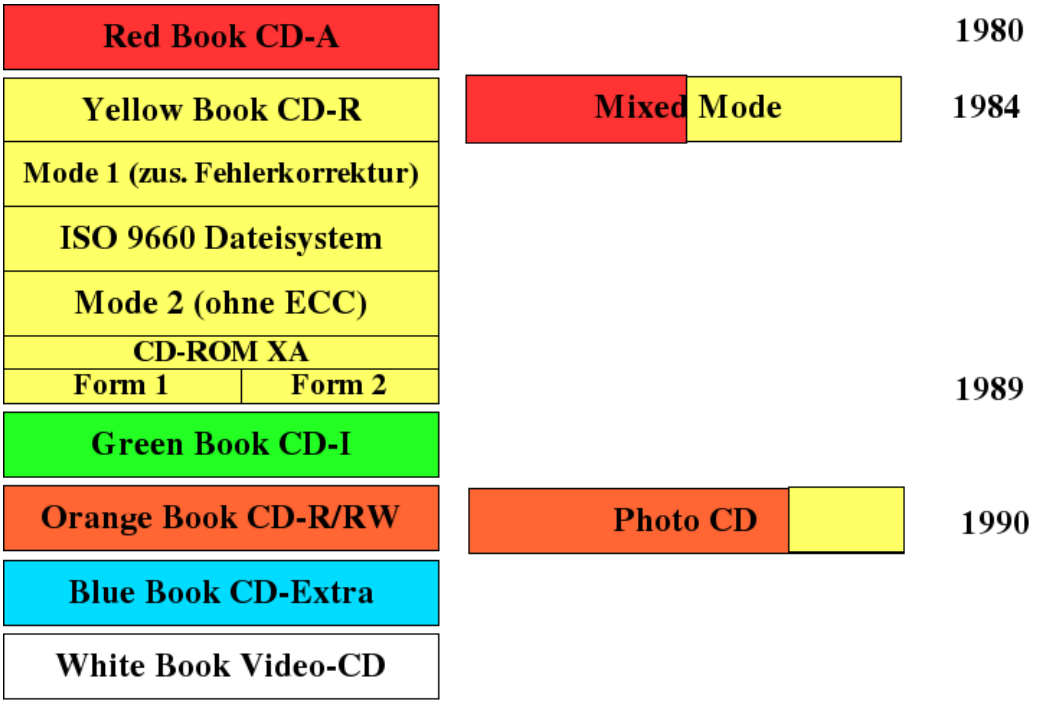



Bild 3.10 CD-Formate

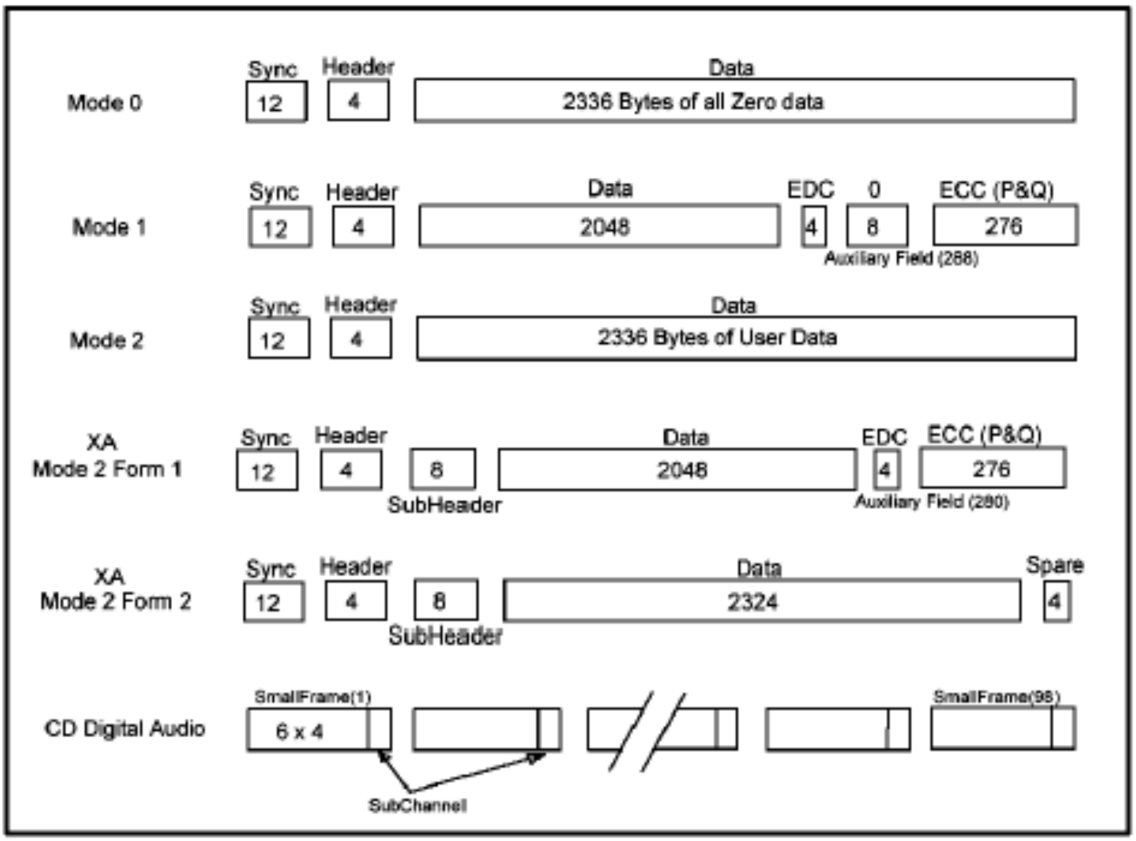



Bild 3.11 Einige CD-Dateiformate

- CD-Datenformat ISO 9660
  - zunächst entsprechend DOS (FAT) entwickelt
  - Dateinamen 8+3 Zeichen
  - Daten liegen linear vor
    - ab Level 3 auch fragmentierte Daten

- Erweiterung im Level 2
  - Dateinamen auf 32 Zeichen erweiter
  - Windows-Erweiterung (Joliet)
    - Dateinamen auf 64 Zeichen erweitert
    - ebenfalls von Linux unterstützt

  - Linux-Erweiterung (Rock-Ridge)
    - Linux-Dateiattribute eingeführt
    - symbolische Links

- bootfähige CD-ROMs

  - 'EL-Torito'-Spezifikation von Phoenix und IBM (1994)

  - Idee: BIOS ersetzt Laufwerk A durch CD-ROM
    - basierend auf ISO-9660 Dateisystem
    - Sektor 16 für Primary Volume Desciptor (enthält FAT)
    - Sector 17 als Boot Volume Descriptor
    - mehrere Boot-Sektoren pro CD-ROM erlaubt
    - Zugriff über Standard BIOS INT-13

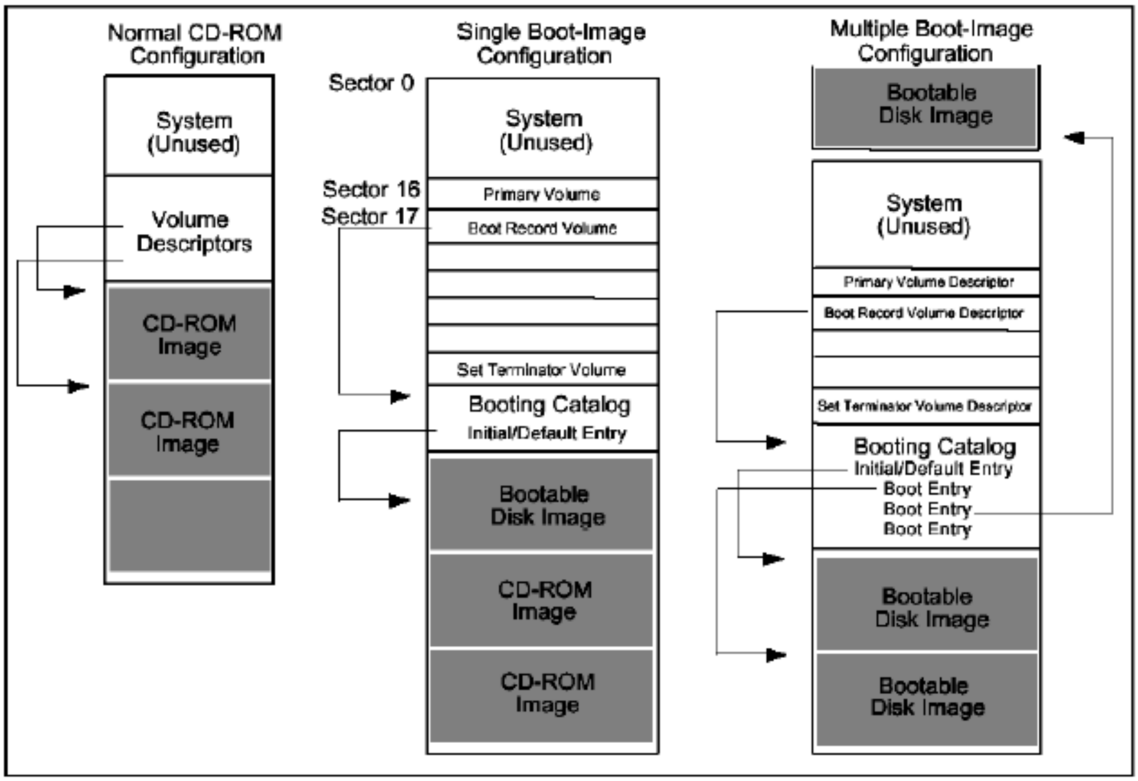



Bild 3.12 Bootfähige CD-ROM

- Erweiterungen zu DVD (digital versatile disk)
- veränderte Headerstrukturen
- Idee: BIOS ersetzt Laufwerk A durch CD-ROM
- weniger Parity-Bits

- Weglassen der Subcodes

DVD-Formate

| | | |
|---|---|---|
| DVD-5 | Single Layer | 4,7 GByte |
| DVD-9 | Double Layer | 9,4 GByte |
| DVD-10 | Double Side | 9,4 GByte |
| DVD-18 | Double Layer, Double Side | 18,8 GByte |

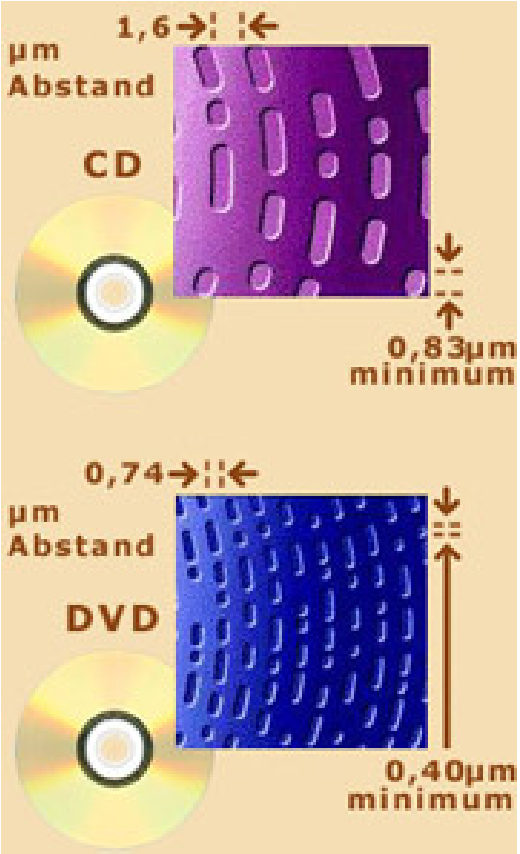



Bild 3.13 Vergleich CD / DVD

# Kapitel 4

# Bussysteme und Schnittstellen

## 4.1 Serielle Datenübertragung (Com-Port)

In modernen Rechnersystemen gibt es eine Vielzahl von Schnittstellen und einige wichtige Bussysteme. Bei den Bussystemen kann man zwischen rechnerinterne Bussysteme (wie Front Side Bus, APG-Port oder PCI-Port) und rechnerexternen Bussystemen unterscheiden. Rechnerexterne Bussysteme, die in unseren Betrachtungen im Vordergrund stehen, dienen dazu, periphere Komponenten an den Rechner anzuschließen. Als einer der wichtigsten Busssyteme der diese Forderung erfüllt hat sich der USB-Bus etabliert. Neben diesem seriellen Bus gibt es auch noch eine weitere serielle Schnittstellen, das COM-Port. Beide Schnittstellen bzw. Bussysteme werden in diesem Kapitel erläutert. Auch die im Rechner häufig anzutreffende parallele Schnittstelle (Drucker-Port oder LPT-Port) wird erläutert.

**serielle Datenübertragung**

- im Rechner vorliegende Daten sind grundsätzlich parallel
- kleinste parallele Einheit -> Byte
- Kern jeder seriellen Datenübertragung -> zeitlich serielle Übertragung der parallelen Daten mit Wandlung von parallel in serielle und umgekehrt
- neben den Daten muss das zugehörige Taktsignal bereitstehen
- Behandung des Taktsignals unterscheidet die verschiedene Arten der seriellen Datenübertragung

  - asynchrone serielle Datenübertragung
    - nur Übertragung von Daten
    - Taktsignal wird von zwei unabhängigen Taktgeneratoren bereitgestellt
    - typisches Beispiel -> COM-Port

  - synchrone serielle Datenübertragung
    - Übertragung von Daten und Taktsignal
    - Typisches Beispiel -> serielle Schnittstelle bei Mikrocontroller

  - serielle Datenübertragung mit Taktrückgewinnung
    - Übertragung von Daten
    - typisches Beispiel USB-Bus, CAN-Bus

**asynchrone serielle Datenübertragung**

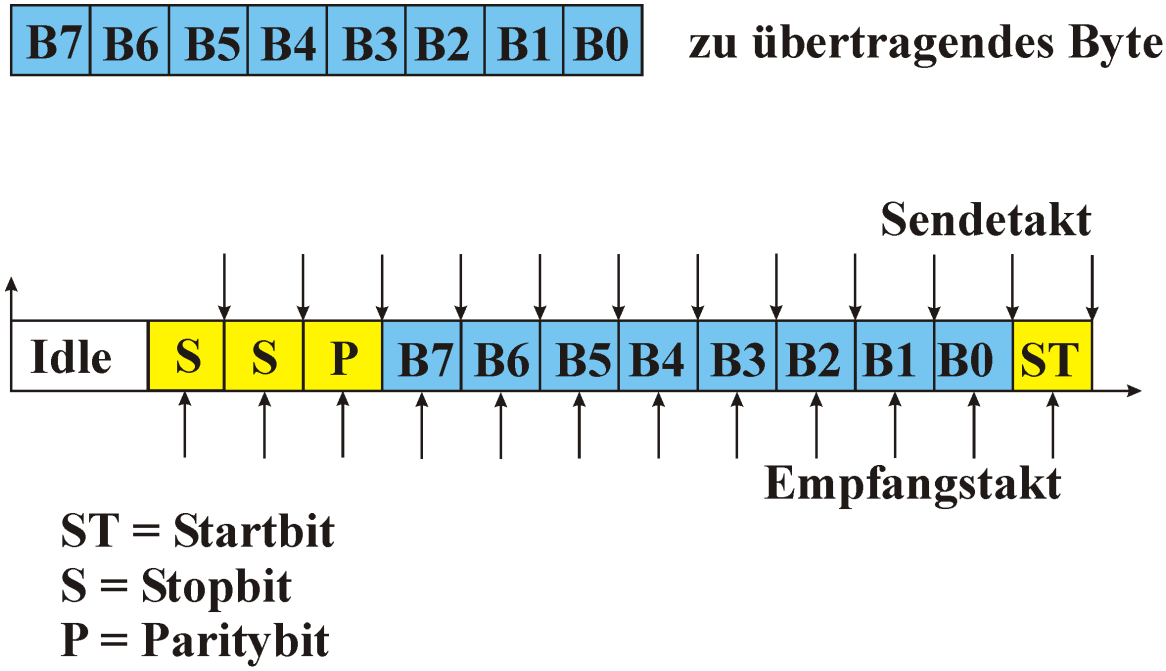



Bild 4.1 Serielle asynchrone Datenübertragung

- Sende- und Empfangstakt werden von zwei unabhängigen Taktgeneratoren bereitgestellt
  - Angabe der Taktfrequenz in 'Baud' (Übertragung von Bits/s)

| Baud Rate |
|---|
| 110 |
| 150 |
| 300 |
| 600 |
| 1200 |
| 2400 |
| 4800 |
| 9600 |
| 19200 |
| 38400 |

- weiterhin Angabe der Anzahl Stopbits und Verwendung des Paritybits erfoderlich
  - damit Bestimmung der Datenübertragungsrate möglich

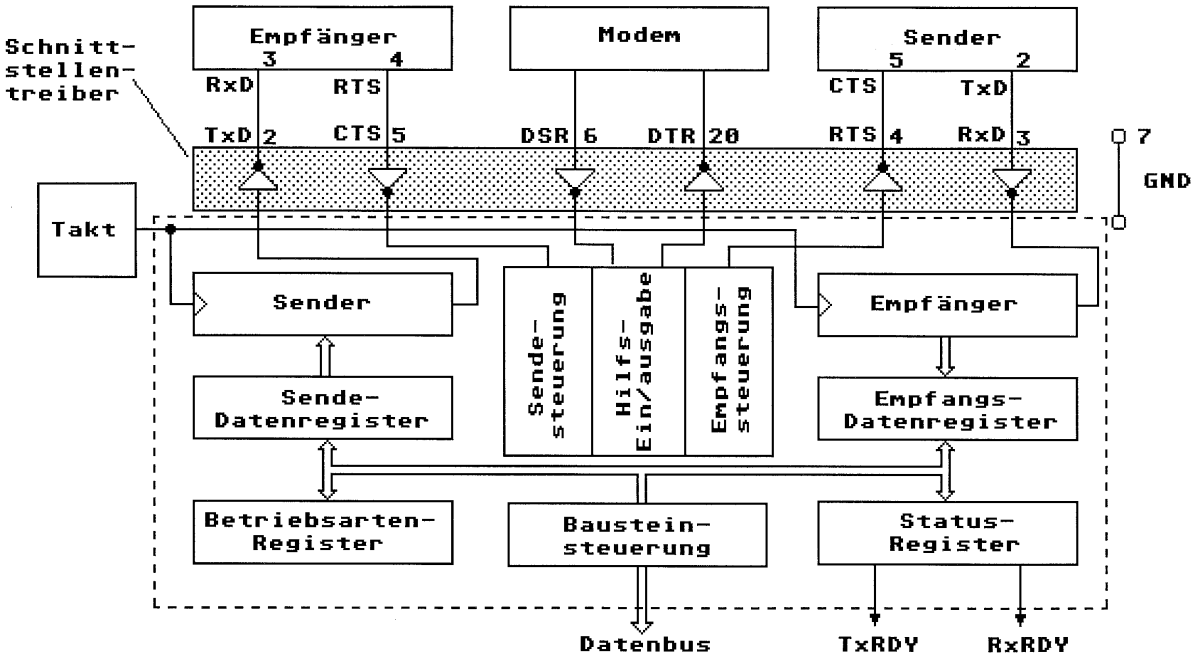



Bild 4.2 Blockschaltbild eines Schittstellenbausteins

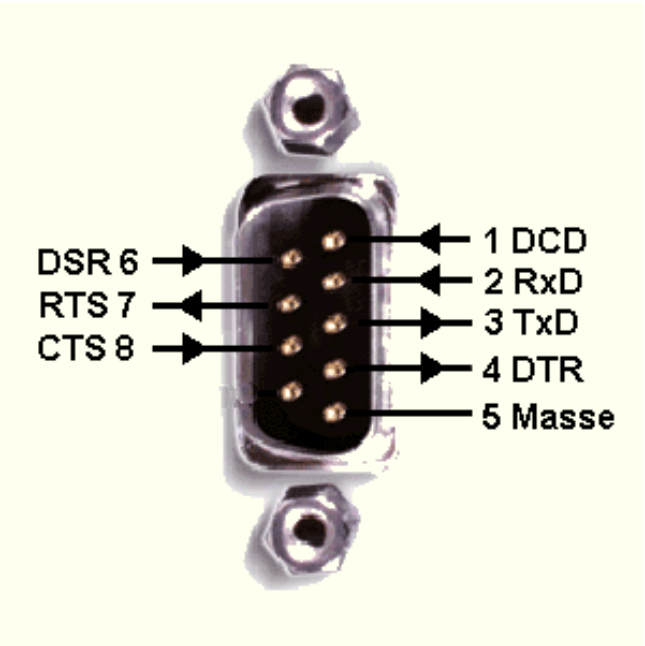



Bild 4.3 Belegung einer seriellen Schnittstelle

| Pin | Bedeutung |
|---|---|
| TxD | serielle Sendedaten (transmit data) |
| RxD | serielle Empfangsdaten (receice data) |
| CTS | Senderfreigabe (clear to send) |
| RTS | Sendeteil einschalten (request to send) |
| DSR | Betriebsbereitschaft eines Modem (data set ready) |
| DTR | Datenendeinrichtung bereit (data terminal ready) |
| GND | Masseverbindung |

- für Ausgangsleitungen von seriellen Schnittstellen sind meist Treiberbausteinen versehen (z.B. RS 232 Treiber am COM-Port)

- Verwendung von Spannungpegel
  - Bit = 1 (Mark) -> Spannungspegel -3V bis -12V
  - Bit = 0 (Space) -> Spannungspegel +3V bis +12V

- in Automatisierungsanwendungen auch eingeprägte Ströme gebräuchlich
  - Bit = 1 -> 20mA
  - Bit = 0 -> 0 oder 4mA

## 4.2 Serielle Datenübertragung (USB)

- serielle Datenübertragung mit Taktrückgewinnung

  - Verzicht auf Übertragung des Taktsignals
    - nur Übertragung von Daten
    - Taktsignal in Datenbits enthalten
    - Erzielung größerer Datenübertragungsraten beabsichtigt

### Allgemeine Eigenschaften des USB

- rechnerexternes Bussystem (Punkt zu Punktverbindungen)
- bitserielle Datenübertragung
- Unterstützung verschiedener Bandbreiten bei der Datenübertragung
  - Übertragung von Daten für Tastatur / Maus -¿ wenige kBit/s
  - Übertragung von Video/Audio-Daten -> bis 400 MBit/s
  - Datentransfer und Messagestreams unterstützt
- bis zu 127 Schnittstellen (Devices) betreibbar
- unterschiedliche Topologien abbildbar (hierachisches System mit Hub, Host und Devices)
- Identifizierung der Busteilnehmer möglich
- einfache Gestaltung von Schnittstellen unf Verbindungskabel
- rekonfigurierbar während des laufenden Betriebs

| PERFORMANCE | APPLICATIONS | ATTRIBUTES |
|---|---|---|
| **LOW-SPEED**<br>• Interactive Devices<br>• 10 – 100 kb/s | Keyboard, Mouse<br>Stylus<br>Game Peripherals<br>Virtual Reality Peripherals | Lowest Cost<br>Ease-of-Use<br>Dynamic Attach-Detach<br>Multiple Peripherals |
| **FULL-SPEED**<br>• Phone, Audio, Compressed Video<br>• 500 kb/s – 10 Mb/s | POTS<br>Broadband<br>Audio<br>Microphone | Lower Cost<br>Ease-of-Use<br>Dynamic Attach-Detach<br>Multiple Peripherals<br>Guaranteed Bandwidth<br>Guaranteed Latency |
| **HIGH-SPEED**<br>• Video, Storage<br>• 25 – 400 Mb/s | Video<br>Storage<br>Imaging<br>Broadband | Low Cost<br>Ease-of-Use<br>Dynamic Attach-Detach<br>Multiple Peripherals<br>Guaranteed Bandwidth<br>Guaranteed Latency<br>High Bandwidth |

Bild 4.4 Anwendungsbeispiele

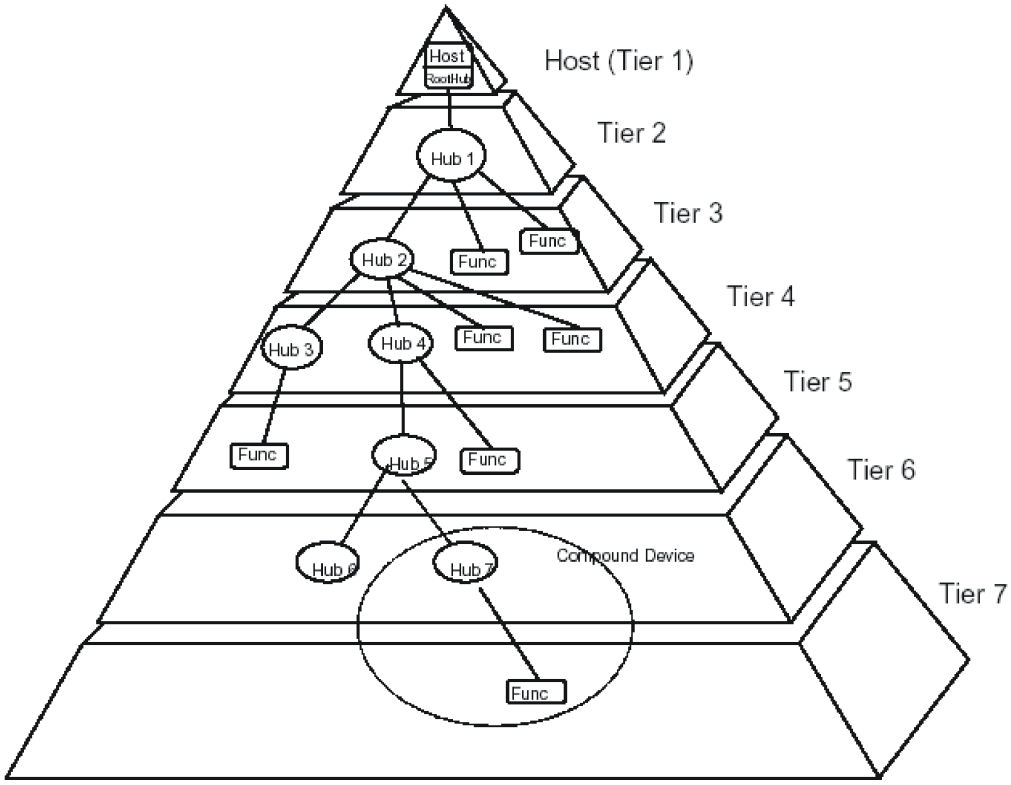



Bild 4.5 Hierachische (Stern)-Topologie des USB

## Schnittstellen des USB

- USB-System startet mit dem Host

- zwischen dem Host und den Devices können Hub‘s integriert sein
- die Adressierung erfolgt einheitlich durch Angabe von Portadressen (insgesamt 127 verschiedene möglich)
- elektrische Übertragung der Daten als Differenzsignale
- Versorgung des Devices (z. B. Self-powered Hub) kann über die Schnittstelle erfolgen

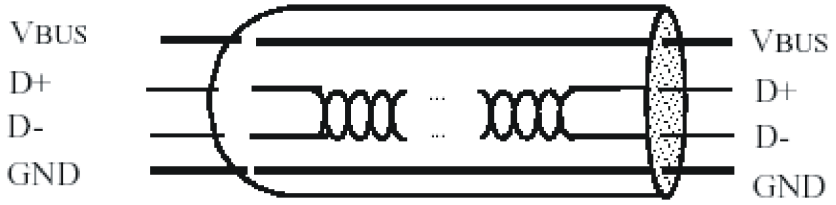



Bild 4.6 USB-Kabel

- USB unterscheidet zwischen zwei mechanischen Schnittstellen-Definitionen

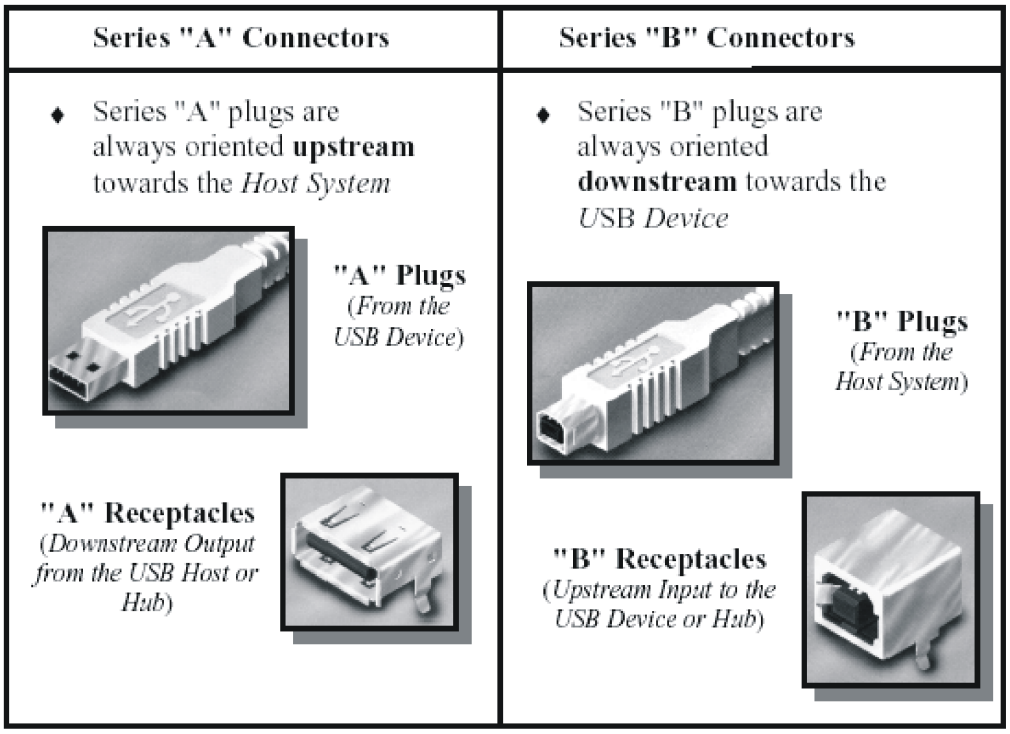



Bild 4.7 USB-Schnittstellen

## Datenübertragung mittels des USB

- elektrische Übertragung der Daten als Differenzsignale
- keine Übertragung des Taktes -¿ Rückgewinnung des Taktes aus der Kodierung der Daten
- Verwendung der NRZI (no return to zero inverted (inverted on zero))
  - LSB zuerst übertragen
  - ’Little endian coding’ verwendet
  - jede Datenübertragung startet mit einem ’Sync-pattern’

- Datenübertragung erfolgt in Paketen, die mit mit einem Identifier (PID) versehen sind
- zeitlich betrachtet werden die Pakete in Frames gruppiert
  - Full/Low-Speed Frame -> Dauer ca. 1ms
  - High-Speed Frame -> Dauer ca. 125 $\mu$s

## Datenkodierung

- Taktrückgewinnung erforderlich

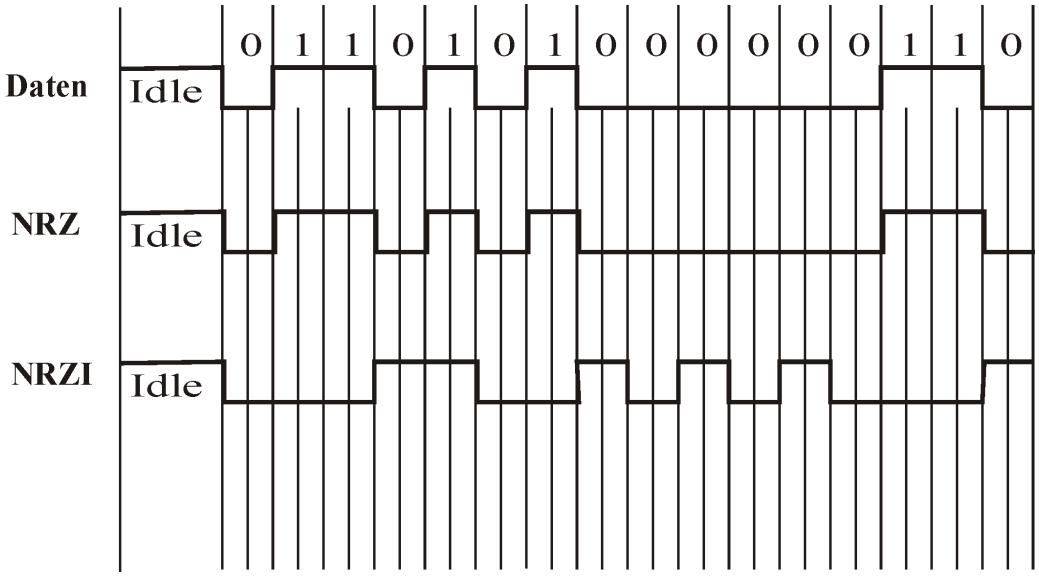



Bild 4.8 Datenkodierung

- bei einer Folge von mehr als 6 'one' -> Integration eines 'zero' notwendig (bitstuffing)

## Übertragung der Daten

- Daten in Frames übertragen

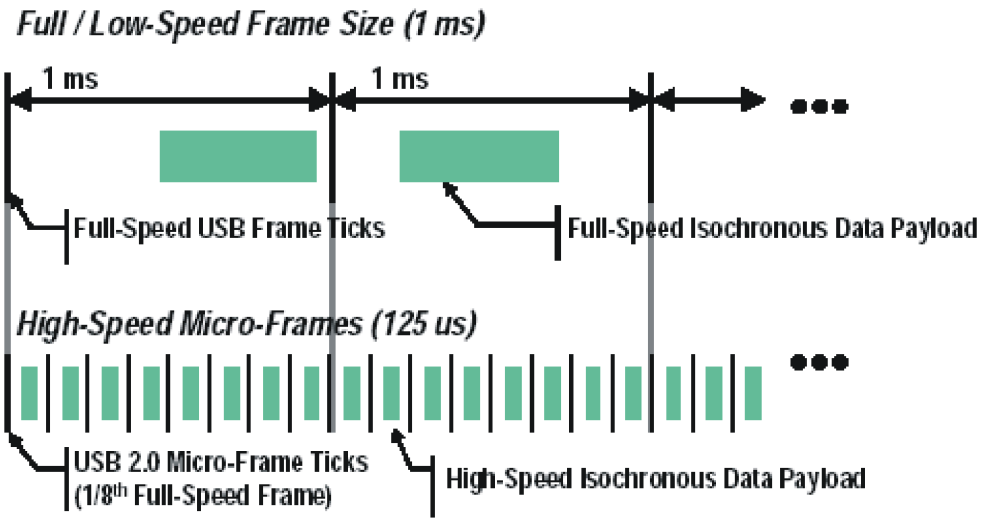



Bild 4.9 Übersicht der Datenpakete

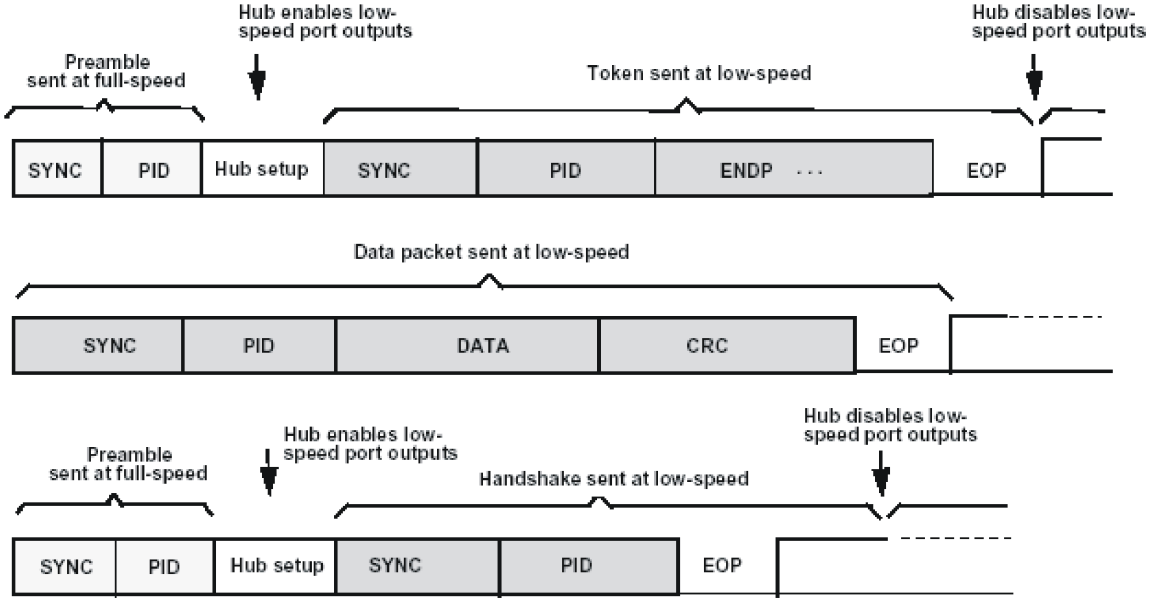
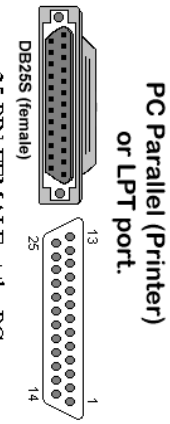



Bild 4.10 Datenpakete mit Steuerinformationen

## 4.3 Parallele Datenübertragung

- parallele Datenübertragung erfolgt meist byteseriell (parallele Übertragung von 8 Bits)
- anstatt eines Taktsignals (Gültig-Meldung) werden 'Strobe'- oder 'Handshake'-Signale verwandt
- die Richtung des Datentransfers
  - ist durch die verwendete Schnittstelle festgelegt (z. B. bei LPT-Port)
  - oder programierbar (z.B. bei Microcontrollern)




25 PIN FEMALE at the PC.
BucK4ABT

| Pin | Name | Dir | Description |
|---|---|---|---|
| 1 | /STROBE | ↓ | Strobe |
| 2 | D0 | ↓ | Data Bit 0 |
| 3 | D1 | ↓ | Data Bit 1 |
| 4 | D2 | ↓ | Data Bit 2 |
| 5 | D3 | ↓ | Data Bit 3 |
| 6 | D4 | ↓ | Data Bit 4 |
| 7 | D5 | ↓ | Data Bit 5 |
| 8 | D6 | ↓ | Data Bit 6 |
| 9 | D7 | ↓ | Data Bit 7 |
| 10 | /ACK | ↑ | Acknowledge |
| 11 | BUSY | ↑ | Busy |
| 12 | PE | ↑ | Paper End |
| 13 | SEL | ↑ | Select |
| 14 | /AUTOFD | ↓ | Autofeed |
| 15 | /ERROR | ↑ | Error |
| 16 | /INIT | ↓ | Initialize |
| 17 | /SELIN | ↓ | Select In |
| 18 | GND | — | Signal Ground |
| 19 | GND | — | Signal Ground |
| 20 | GND | — | Signal Ground |
| 21 | GND | — | Signal Ground |
| 22 | GND | — | Signal Ground |
| 23 | GND | — | Signal Ground |
| 24 | GND | — | Signal Ground |
| 25 | GND | — | Signal Ground |

Bild 4.11 Belegung LPT-Port

- Programmierung des LPT-Ports durch Belegung zugehöriger Register

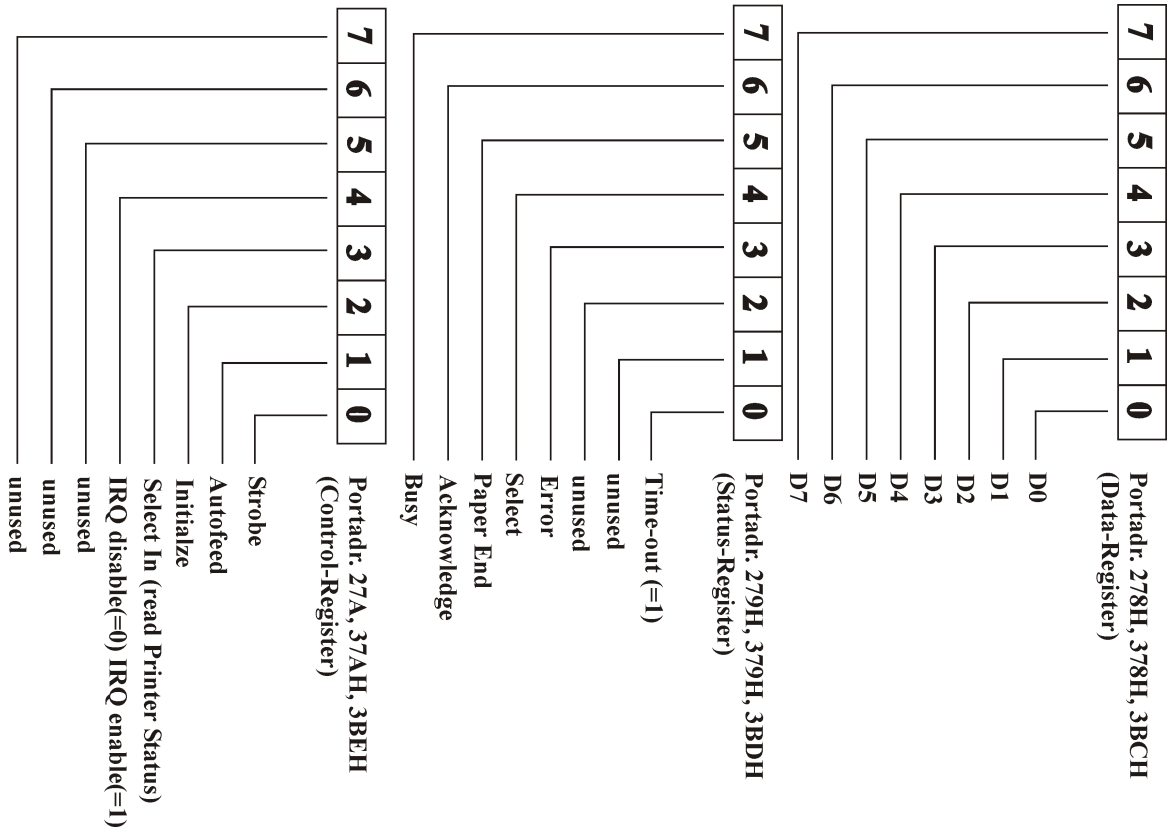



Bild 4.12 Register der LPT-Ports

## 4.4 Schnittstellen Motherboard

- Verarbeitung und Transfer der CPU-Signale durch 'ChipSet'
- Zusammenfassung von Signalen zur Ansteuerung von Speicher und Schnittstellen
- Northbridge
  - Bereitstellung von Schnittstellen (oder Bus) zur Ansteuerung des Arbeitsspeichers (FSB- Front Side Bus mit reduzierter Taktfrequenz z.B. FSB866 oder FSB1066)
  - typischer Chipset P965 (Intel)
- Southbridge
  - Bereitstellung nahezu aller weiteren Schnittstellen im Rechner
  - Anschlüsse von Harddisc (SATA-Schnittstelle)
  - PCI-Schnittstellen (in Standard und Express-Version)
  - Bereistellung von USB-Ports
  - über 'Lowlevel-Chipset' SuperI/O Bereitstellung von Com-Ports, LPT-Schnitt.
  - typischer Chipset ICH8R (Intel)

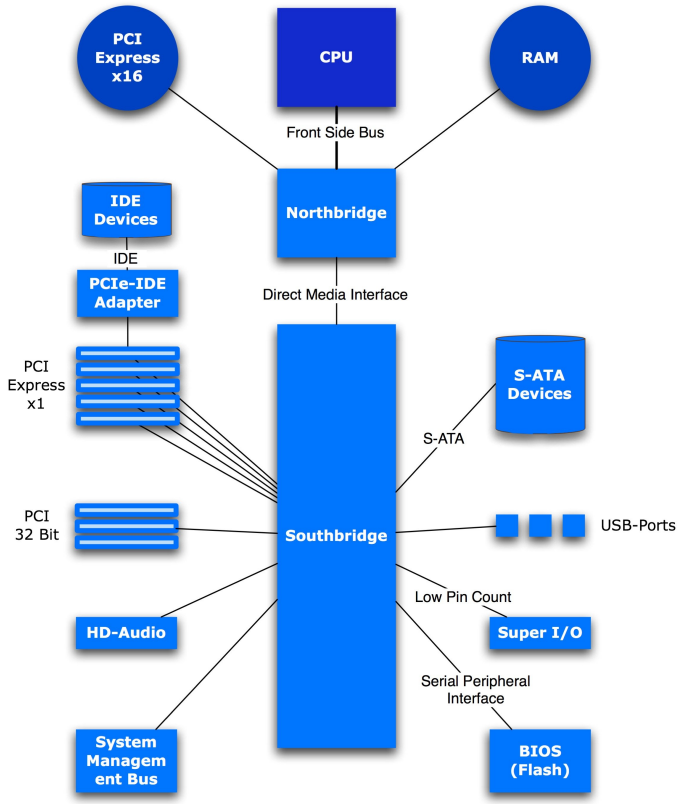



Bild 4.13 Schnittstellen auf dem Motherboard

# Kapitel 5

# Einführung in die 8086-Architektur

## 5.1 Registermodell der 8086-Architektur

Die 8086-Architektur ist aus Sicht der Rechnerarchitektur nicht unbedingt durch einen konsistenten oder orthogonalen Aufbau gekennzeichnet. Trotzdem ist gerade diese Architektur die z. Z. bedeutsamste. Der Grund besteht vor allem in der über nahezu allen Generationen von Prozessoren eingehaltenen Kompatibilität. Dies führt dazu das auch in heutigen Systemen (Pentium IV) noch ein Modell für die Programmierung existiert, dass in den 70'Jahren des letzten Jahrhunderts kreiert wurde; eben gerade die 8086-Architektur.
Nun ist es nicht so, dass in modernen Prozessoren (Pentium IV) dieselben Schaltungen verwandt würden wie im 8086. In derartigen Prozessoren wird ein Modell des 8086 nachgebildet. Die Funktionen des 8086 kann man sich als Programmcode vorstellen, der innerhalb des Pentiums bearbeitet wird; man spricht in diesm Fall von einem Mikrocode.

**Registermodell**

- Registermodell der 8086-Architektur in allen Intel-Prozessoren aufwärts von 8086 anzutreffen
- Registermodell ist Grundlage der 16-Bit-Programmierung (auch noch bei der Pentium-Programmierung anzutreffen)
- Erweiterung zur 32-Bit-Programmierung durch Einführung von 32-Bit-Register (Verwendung von Extended-Register, z.B. EAX anstatt AX)
- mit der 8086-Architektur erfolgt die Einführung der Segmentierung des Arbeitsspeichers
  - Addressbreite der 8086-Register umfassen nicht die Adressbreite des Prozessores
  - Segmentierung erfolgt mittels Register und Algorithmus (8086)
  - Segmentierung beim Pentium erfolgt durch Verwendung von Tabelleneinträgen

  - Kennzeichen des Registermodells der 8086-Architektur

    - keine eindeutige Trennung zwischen Address- und Datenregister
    - insgesamt wenig Register
    - Einschränkungen bei der Verwendung der Register für die Quell- und Zieladressierung (beim Speicherzugriff muss ein Operand im Register stehen)

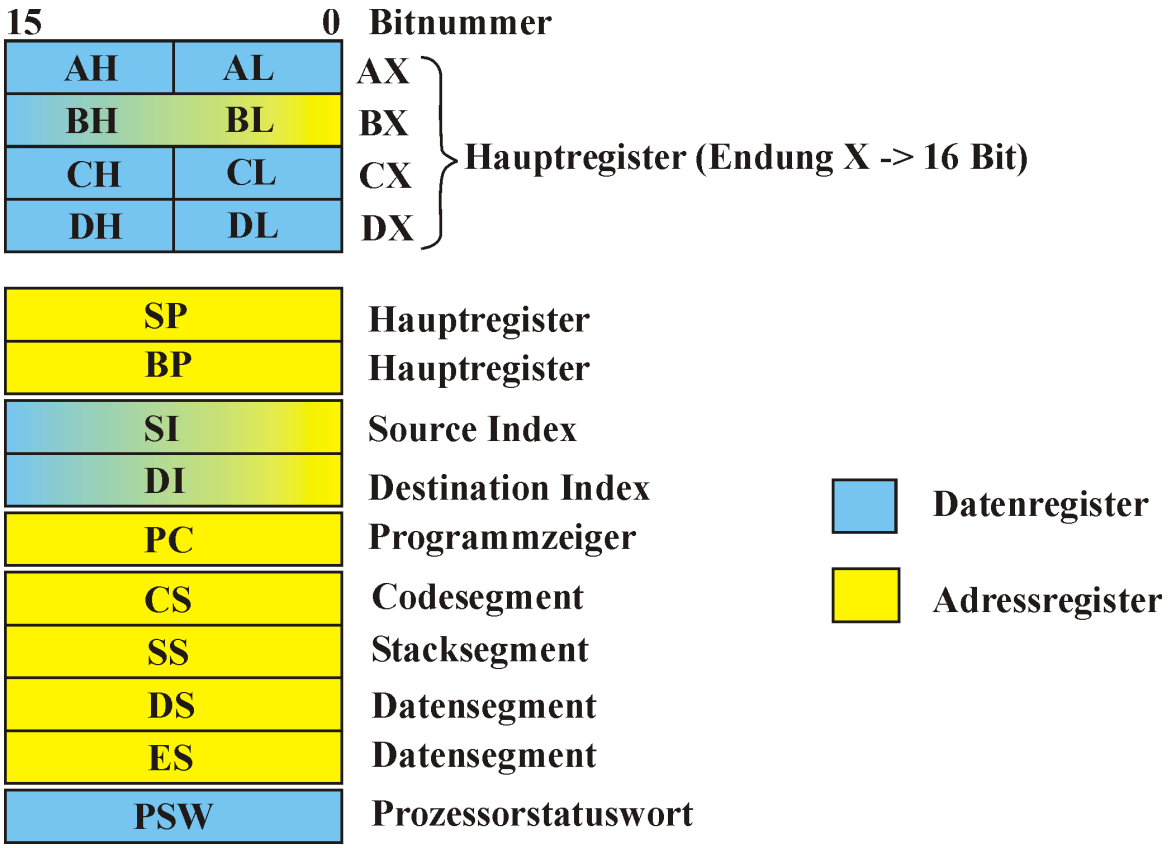



Bild 5.1 Registermodell der 8086-Architektur

**Definition 5.1** Unter *Segmentierung* versteht man bei der Speicherverwaltung die Unterteilung des benutzten Speicheradressraums in einzelne Segmente. Zweck der Unterteilung ist meist die Implementierung von Schutzmechanismen. Je nach Betriebssystem und unterliegender Hardware können einem Segment verschiedene Attribute zugewiesen werden. So können beispielsweise Programm-, Daten- und Stack-Segmente festgelegt werden.

## Registergestützte Segmentierung

- in der 8086-Architektur sind insgesamt 3 Segmente vorgesehen (Programmsegment, Datensegment und Stacksegment)
- für die jeweiligen Segmente werden ein bzw. zwei Segmentregister verwandt
- Segmentregister werden meist impliziet benutzt (Defaulteinstellung für die Segmente)
  - CS-Segmentregister -> Programmsegment
  - DS- und ES-Segmentregister -> Datensegment
  - SS-Segmentregister -> Stacksegment

- Adressierung mit Hilfe der Segmentierung (registergestützt)
  - Bildung einer absoluten Adresse
  - Verwendung des zugehörigen Segmentregisters
  - Verwendung der effektiven Adresse innerhalb des Segments

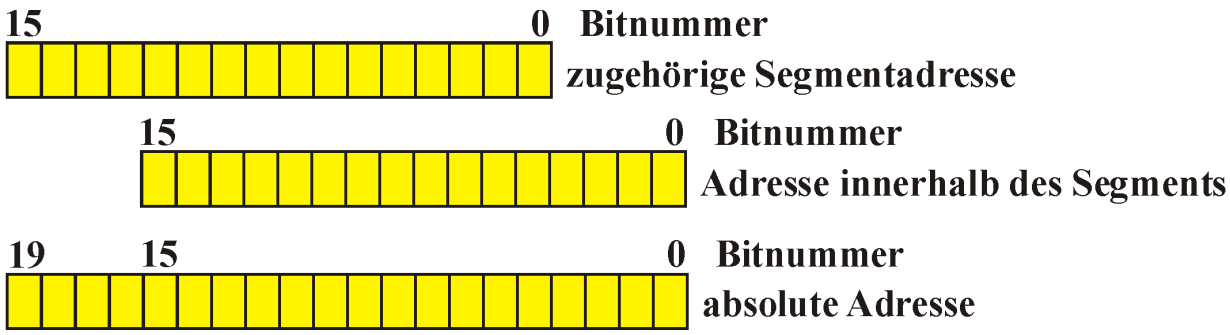



Bild 5.2 Bildung einer absoluten Adresse

**Prozessorstatuswort**

- nach Ausführung eines Befehls -> Änderung des Prozessor-Statuswortes (Statusregister) möglich
- Statusregister wird mit bedingten Sprungbefehlen abgefragt

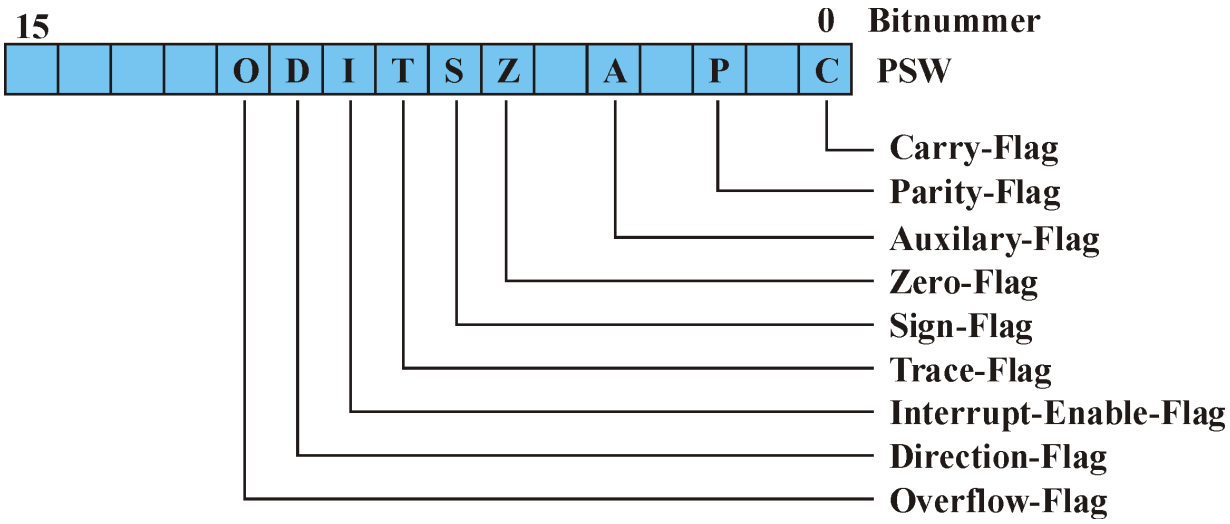



Bild 5.3 Belegung des Prozessorstatuswortes

## 5.2 Maschinenbefehlssatz des Pentium

- Maschinenbefehlssatz (ISA instruction set architecture) setzt auf Registermodell auf
  - Verwendung von Daten- und Adressregister
  - Besonderheit bei 8086 -> Segmentierung
    - impliziete Verwendung der Segmentregister (8086) oder Selectoren (Pentium)
    - Zuordnung der Segmente
      - Programmsegment CS:
      - Datensegment DS:
      - Stacksegment SS:

- falls die Zuordnung nicht verändert werden soll erfolgt keine Angabe des zugehörigen Segmentes

- Veränderungen sind mit dem Segmentvorsatz möglich (insbesondere sinnvoll beim Datensegment)

## Adressierungsarten

- Verfahren des Zugriffs auf bestimmte Speicherstellen
- für den Zugriff auf Daten und den Zugriff auf Programmstellen bedeutsam (beim Begriff ?Zugriff? wird nicht zwischen einem Lesezugriff oder einem Schreibzugriff unterschieden)
- Adressierungsarten lassen sich gruppieren in:
  - Registeradressierung
  - Wertadressierung (immediate addressing)
  - direkte Addressierung
  - indirekte Adressierung
  - relative Adressierung
- insbesondere bei den direkten, indirekten und relativen Adressierungen gibt es viele Untervarianten
- In der Hochsprache C sind die einzelnen Adressierungsvarianten versteckt (aber natürlich eingesetzt!)

**Definition 5.2** Die *Registeradressierung* plaziert Quell- und/oder Zieloperanden in Register des Prozessors. Ein Adressrechenwerk ist für diese Adressierung nicht erforderlich.

## Registeradressierung

- einfachste Adressierungsvariante
- in jedem Prozessor anzutreffen
- der oder die Operanden stehen in einem oder zwei Register
- z. B. MOV AX,BX ; Inhalt von BX nach AX transferieren

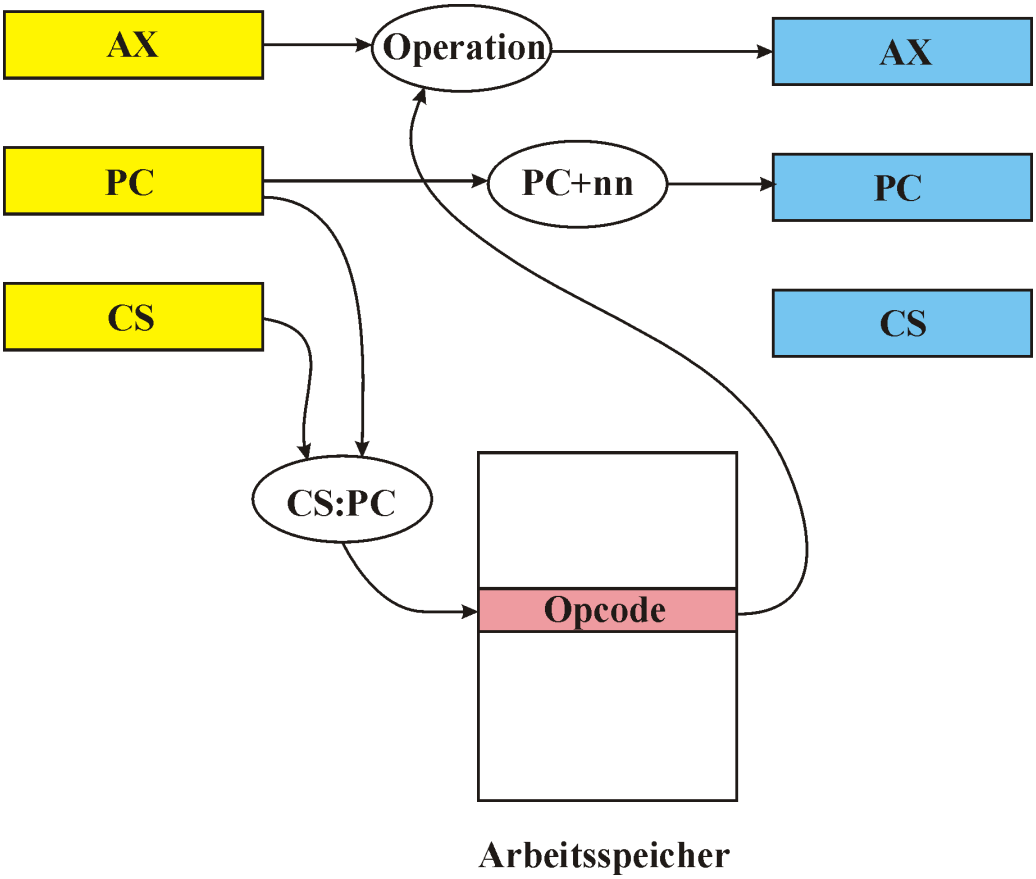


Bild 5.4 Registeradressierung

**Definition 5.3** Bei der *Wertadressierung* wird der zu programmierende Wert im Programmsegment plaziert. Die Wertadressierung kann sich nur auf eine Quelle beziehen. Von Programmierwerkzeugen wird die Qulle als Konstante behandelt.

## Wertadressierung

- Wert (Konstante) steht im Befehl
- z. B. ADD AX,1234H ;Summe AX plus 1234H in AX

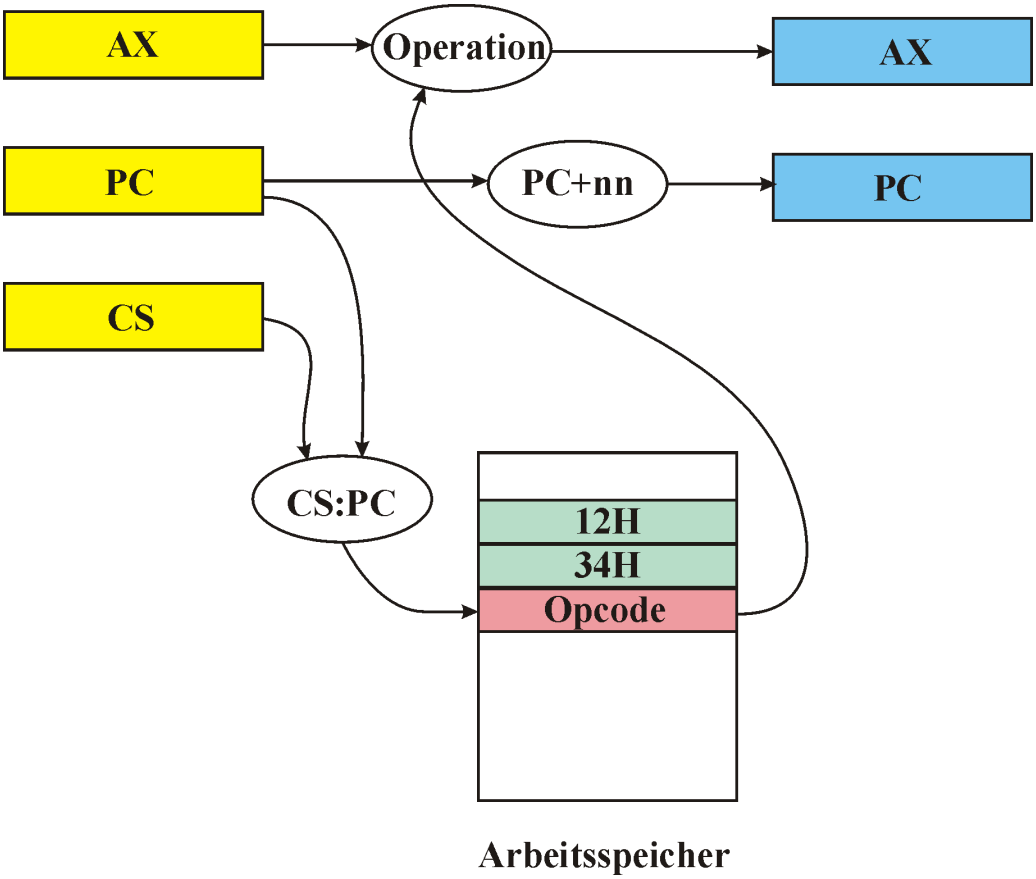



Bild 5.5 Wertadressierung

**Definition 5.4** Bei der *direkten Adressierung* wird die Adresse im Programmsegment plaziert. Die direkte Adressierung kann auf Quell- und Zieloperanden angewendet werden. Bedingt durch die Platzierung der Adresse im Programmsegment ist diese während des Programmablaufs nicht mehr änderbar.

### Direkte Adressierung

- Adresse des Operanden steht im Befehl
- in der Hochsprache C für globale Variablen eingesetzt
- z. B. MOV AX,[2034H] ;Inhalt der Speicherstelle 2034H in AX
- Anzahl der Bytes hängt vom Registernamen ab

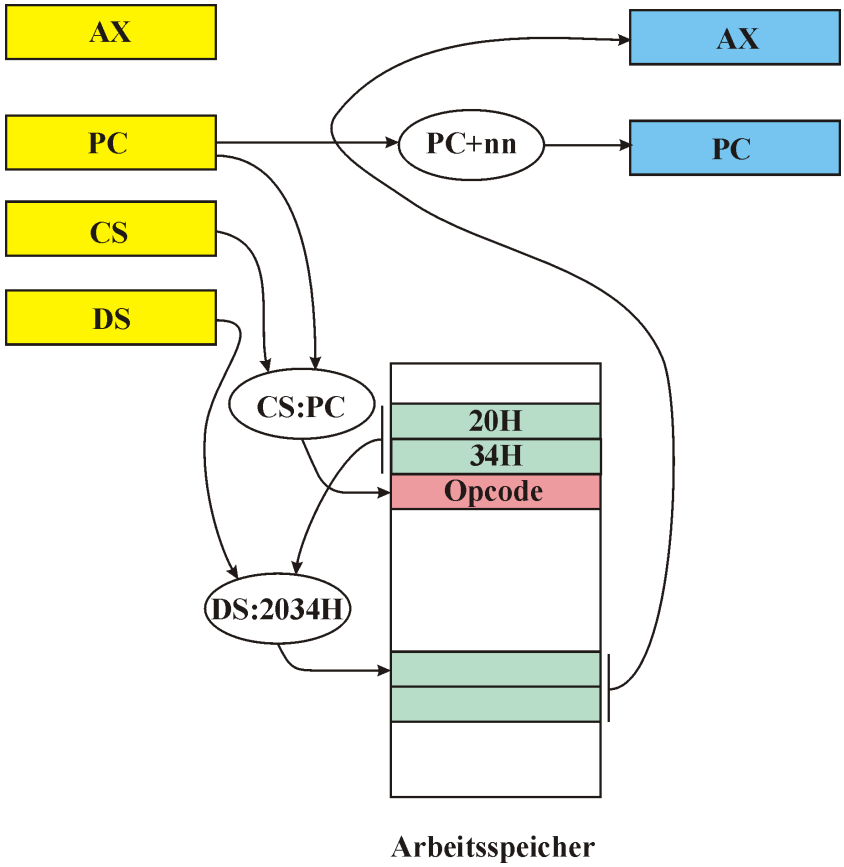


Bild 5.6 Direkte Adressierung

**Definition 5.5** Bei der *indirekten Adressierung* wird die Adresse in einem Adressregister platziert und kann dadurch während der Laufzeit des Programms abgeändert werden. Die indirekte Adressierung ist auf Quell- und Zieloperanden anwendbar.

## Indirekte Adressierung

- Adresse des Operanden steht im Adressregister
- in der Hochsprache C für berechnete Variablen eingesetzt
- z. B. MOV AX,[BX] ;Inhalt Speicherstelle auf die BX in AX
- Anzahl der Bytes hängt vom Registernamen ab

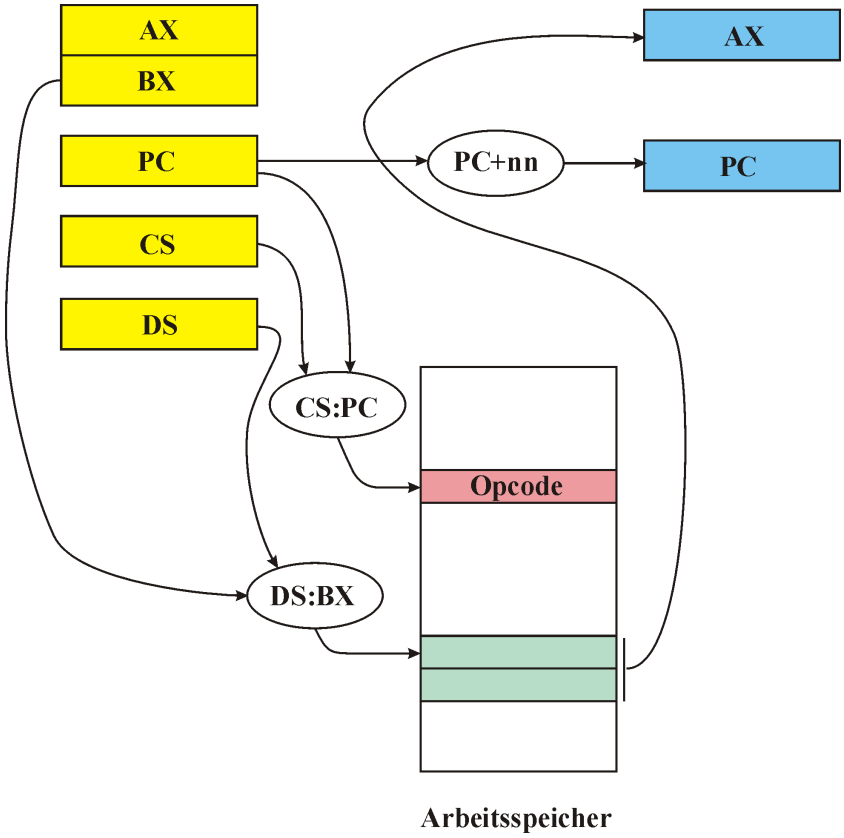


Bild 5.7 Indirekte Adressierung

**Definition 5.6** Bei der *relativen Adressierung* wird die Adresse relativ zum Programmzeiger angegeben. Durch diese Adressierung lassen sich positionsunabhängige Programme entwickeln.

## Relative Adressierung

- Adresse des Operanden wird relativ zum Inhalt des PC angegeben
- beim 8086 nur für Adressierung von Sprungzielen vorgesehen (in Sprungbefehlen)
- relative Adressierung von Daten nicht möglich (das Register PC kann nicht als Adressregister eingesetzt werden)
- prinzipiell ermöglicht die relative Adressierung positionsunabhängige Programme
- z. B. JZ dis ;Springe wenn Zero-Flag = 0 an die Stelle PC +/- dis

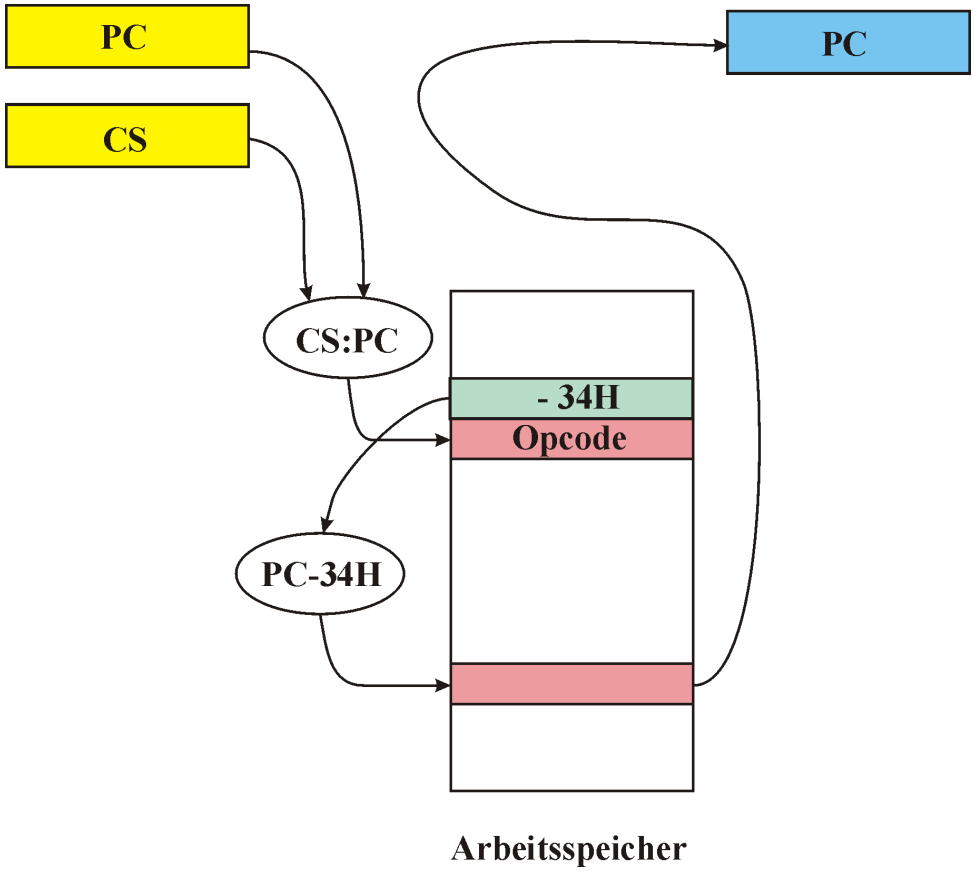



Bild 5.8 Relative Adressierung

## Befehlssatz

- Bildung von logischen Befehlsgruppe (nur für den Programmierer sinnvoll)
- Zusammenfassung der wichtigsten Befehle:
  - a) Transferbefehle
  - b) Arithmetische Befehle
  - c) Logische Befehle
  - d) Bitinstruktionen
  - e) Sprung- und Call-Befehle
  - f) Andere

# Kapitel 6

# 8086-Architektur im Pentium

## 6.1 Begrenzungen der 8086-Architektur

Wie schon zuvor erwähnt (Kap. 5) stellt die 8086-Architektur einen Teil des Pentiums dar. Diese Architektur ist als Kompatibilitätsgründen im Pentium zu finden. In diesem Kapitel wollen wir uns zunächst noch einmal die Begrenzungen der 8086-Architektur verdeutlichen, die besonders auffallend im klassischen Prozessor 8086 sichtbar werden.
Nach diesen Ausführungen werden wir dann die Funktionen des Pentiums beleuchten, die ihn als modernen Prozessor kennzeichnen und in jedem Rechnersystem als notwendig angesehen werden.

- die Kenntnis des Programmiermodells und die Verwendung des Statusregister reicht meist für die Programmierung
  - Schwierigkeit sind jedoch dann zu erwarten, wenn Adressbreite und Breite des Addressbusse differieren
  - in 8086-Architektur durch Segmentierung behoben
  - Segmentierung unterscheidet drei Segmenten
  - Programmsegment
    - Adresse durch CS:IP
  - Stacksegment
    - Adresse durch SS:SP
    - Adresse durch SS:BP
  - Datensegment
    - Adresse durch DS:Adressierung innerhalb des Segmentes
    - Adresse durch ES:Adressierung innerhalb des Segmentes
  - beim 8086 können auch 'fremde' Segmentregister verwandt werden (Segmentvorsatz) (nicht beim 80386 - Pentium)

### Adressraum und Segmentierung

- Adressbus 8086 -> 20 Bit -> Adressraum 1MByte
- Adressregister 8086 -> 16Bit -> Adressierung von 64 Kbyte
- Folgerung -> Speicherbereich innerhalb des Segmentes mehrfach im Adressraum des 8086 vorhanden

- Problemlösung:
  - Programm, der Stack und die Daten innerhalb von 64KByte
  - Adressierung nur innerhalb des Segmentes (NEAR-Adressierung Programm, Stack und Daten)
  - Segment kann durch Segmentvorsatz beliebig im Speicherbereich des 8086 verschoben werden
  - Modell TINY

- Programm in 64 KByte, Stack und Daten in weiteren 64KByte
  - Adressierung nur innerhalb des Segmentes (NEAR-Adressierung Programm, Stack und Daten)
  - CS-Segment unterscheidet sich von SS- und DS- oder ES-Segment
  - Modell SMALL

- Programm in mehr als 64 Kbyte, Stack in 64 Kbyte, Daten in mehr als 64 Kbyte
  - Adressierung außerhalb des Segmentes für Programm und Daten (FAR-Adressierung, das jeweils zugehörige Segmentregister wird auch verändert)
  - CS-Segment und DS- oder ES-Segment werden während der Laufzeit des Programms verändert
  - Modell LARGE

- Auswahl der Modelle durch Compiler-, oder Assemblereinstellung

- Adressraum des 8086 kann nicht mit Attributen versehen werden
  - keine Einteilung in einen Supervisor- und Usermode möglich
  - trotz Segmentierung erfolgt keine Vergabe von Zugriffrechten
  - 8086 läuft praktisch immer im höchstpriorsierten ?Kernelmode?
  - durch Programm-Irrläufer leicht Absturz des Gesamtsystems

- der Adressraum des 8086 ist eingeteilt in:
  - Speicherbereich (1 Mbyte)

  **Beispiel 6.1** MOV AH, [1000H] ;Lade den Inhalt Speicherstelle 1000 in AH

  - I/O-Bereich (64 Kbyte)

  **Beispiel 6.2** IN AL,[DX] ;Lade Inhalt der I/O-Adresse auf die DX zeigt in AL

### Adressierungsvarianten

- Einschränkungen bei den Adressierungsvarianten

- keine relativen Adressierungen für Daten (das Register PC kann nur für Programmadressen verwandt werden)
    - positionsunabhängige Programme können nicht direkt entwickelt werden (dieser Nachteil wird durch die Segmentierung aufgehoben)

- ein Speicher- zu Speichertransfer ist nicht möglich (Ausnahme: spezielle Stringbefehle)
    - Transfer muß über mindestens ein Register gehen

- generell stehen sehr wenig Register zur Verfügung
    - häufiger Zugriff auf den Speicher (auch beim Pentium)

## 6.2 Erweiterungen der 8086-Architektur

- Implementation der IA32-Architektur (Intel Architecture 32)
- Breite des Hauptregistersatzes 32 Bit
- jedes Hauptregister für Adressierung des Arbeitspeichers verwendbar
- Adressbreite 32 Bit -> 4 Gbyte Adressraum
- Bereitstellung mehrerer prozessorinterner Rechenwerke
    - Floating Point Unit
    - Verarbeitung von Datenpaketen (MMX- und SSE-Erweiterung)
    - Implementation von zwei Integerrecheneinheiten (U- und V-Pipe)

Die Vergrößerung der Wortbreite des Prozessors von 16 auf 32 Bit stellt zunächst eine klassische Erweiterung der 8086-Architektur dar. Mit der Implementation des Pentiums wurden aber auch ein Teil der Beschränkungen in der Verwendung der Register aufgehoben. So sind alle Hauptregister (EAX, EBX, ECX, EDX, ESI, EDI) als Addressregister für die Adressierung von Daten im Datensegnent geeignet; beim 8086 waren dies nur die Register BX, DI und EI. Die Register EBP und ESP können ähnlich wie beim 8086 für die Adressierung von Daten im Stacksegmet verwendet werden.

Eine weitere Verbesserung der Rechenleistung im Pentium ist durch den Einsatz von Rechenwerken für die Verarbeitung von Datenpaketen (MMX- und SSE-Erweiterung) zu verzeichnen.

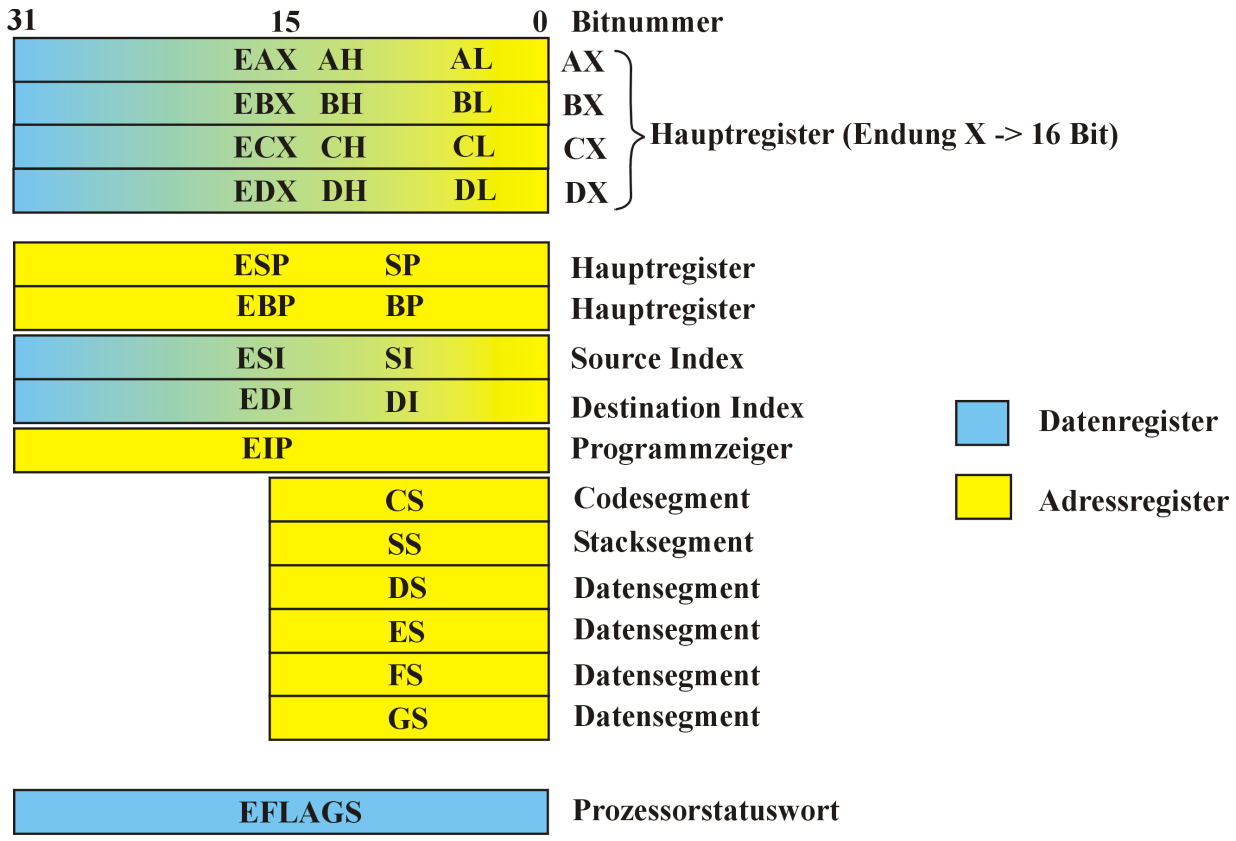



Bild 6.1 Registermodell Pentium (General Purpose)

## Implementation einer FPU (Floating Point Unit)

- Verwendung eines separaten Registersatzes
- Registersatz mit interner Wortbreite von 80 Bit (Register R0 - R7)
- Registersatz kann als Stack betrieben werden (ST(0) - ST(7))
  - Zugriff erfolgt dann grundsätzlich auf den TOP <-> ST(0), (TOP = top of stack)
  - die anderen Register werden entsprechend verschoben
- FPU rechnet mit höherer interner Genauigkeit als externe Floating Point-Zahlentypen erfordern (float -> 32Bit mit 23 Bit Mantisse, double -> 64Bit mit 52 Bit Mantisse)
- Umwandlung der externen Floating Point-Zahlen in interne und umgekehrt erfolgt beim Laden
  - umfaßt eine automatische Rundung beim Speichern einer Flaoting Point-Zahl

- Laden von Zahlen in die FPU-Register (ST(0) - ST(7) erfolgt grundsätzlich aus dem Arbeitsspeicher
  - Verwendung der Adressierungsmöglichkeiten der CPU für das Laden und Speichern der Zahlen der FPU
  - direkter Transfer von Daten zwischen Hauptregister und FPU-Register nicht möglich

- Befehle für die FPU können in die Befehlsfolge der CPU eingestreut sein
  - automatische Trennung der Befehle für CPU und FPU
  - Befehlssatz der FPU umfaßt Lade-, Speicher- und Datenwandlungsbefehle

- Sprungbefehle sind prinzipiell nicht möglich (werden nach Auswertung des Statusregister der FPU von der CPU übernommen)

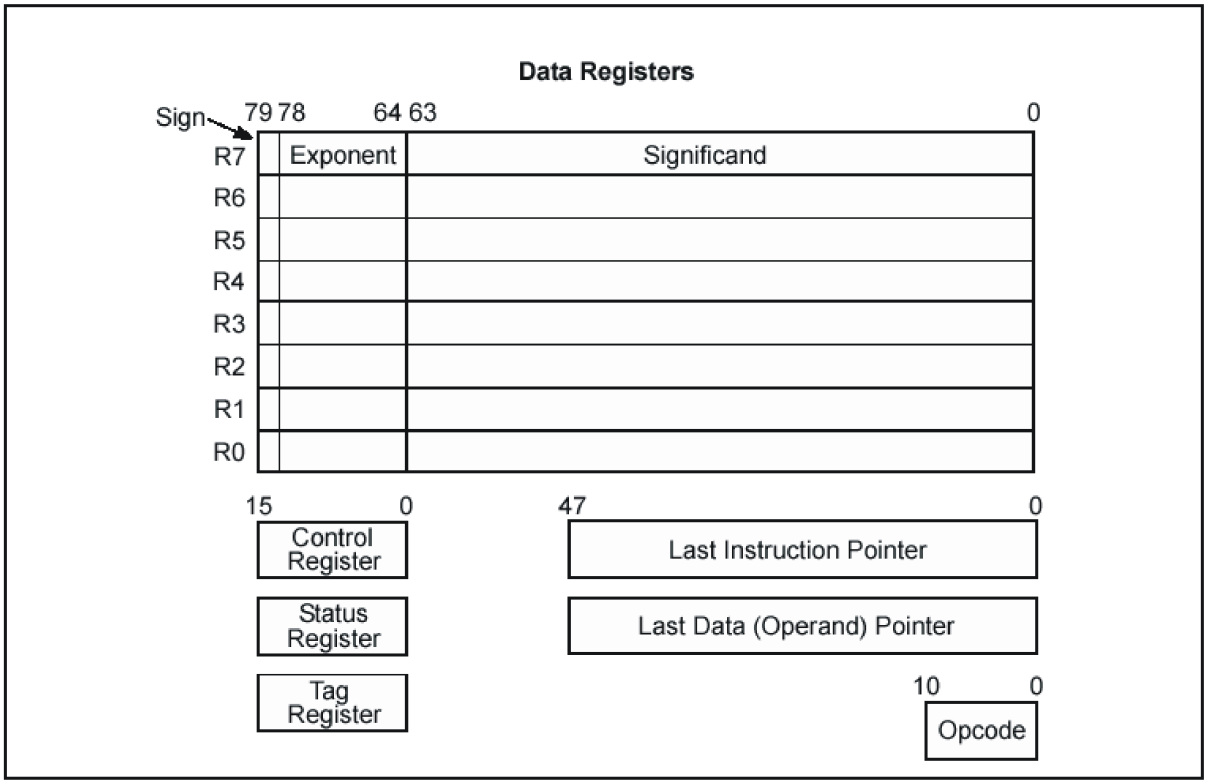



Bild 6.2 Register der FPU

## Implementation von Funktionen zur Verarbeitung von Datenpaketen

- Verarbeitung von Datenpaketen immer dann sinnvoll wenn in einem Register mehrere kleinere Datentypen abgelegt werden können

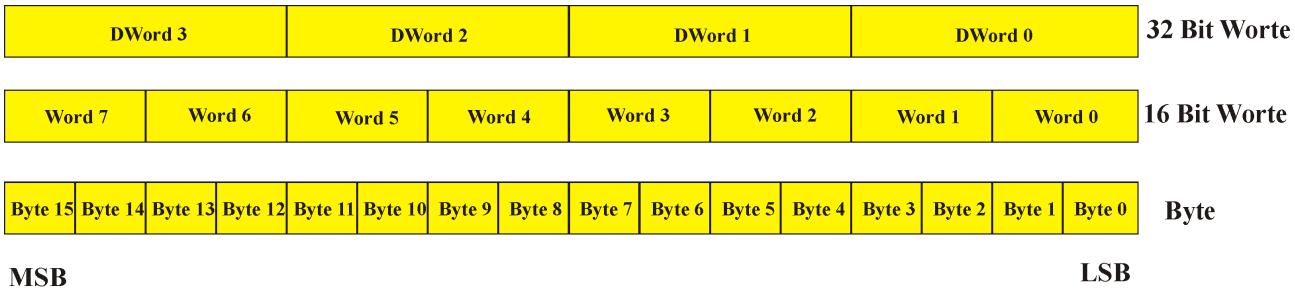



Bild 6.3 Datenpakete in einem 128 Bit Wort (DQWORD)

- Verarbeitung von Datenpaketen in zwei Varianten unterstützt
  - MMX-Erweiterung
  - SSE-Erweiterung

## MMX-Erweiterung

- MMX-Erweiterung (MMX multimedia extension) -> erstmalige Unterstützung der Verarbeitung von Datenpaketen beim Pentium

- Verwendung des Mantissenteils des FPU-Registersatz mit interner Wortbreite von 64 Bit (Register R0 - R7)
- MMX-Erweiterung und FPU können nicht parallel verwandt werden
- damit erhebliche Einschränkung in der Nutzung der MMX-Erweiterung
- Bezeichnung der Register als MM0 - MM7
- als Datenpakete können vorzeichenbehaftete und vorzeichenlose ganzahlige Datentypen mit 8 oder 16 Bit Wortbreite eingesetzt werden
- Implementation eines separaten Befehlssatzes für die Verarbeitung von Datenpaketen
- Laden und Speichern der Datenpakete immer aus dem Arbeitsspeicher
- Adressierung der Datenpakete ähnlich dem Verfahren der FPU

## SSE-Erweiterung

- SSE-Erweiterung (SIMD streaming extension) -> Implementation eines Rechenwerkes zur Unterstützung der Verarbeitung von Datenpaketen ohne Restriktionen wie bei der MMX-Erweiterung

- Verwendung eines sepaten Registersatzes mit 8 Register a 128 Bit
- SSE-Erweiterung und FPU können parallel arbeiten
- Bezeichnung der Register als XMM0 - XMM7
- als Datenpakete können vorzeichenbehaftete und vorzeichenlose ganzahlige Datentypen mit 8,16 oder 32 Bit Wortbreite eingesetzt werden
- Verarbeitung von Floating Point-Zahlen (im 32 Bit float-Format) möglich (die meistgenutzte SSE-Anwendung!)
- Implementation eines separaten Befehlssatzes für die Verarbeitung von Datenpaketen
- Laden und Speichern der Datenpakete immer aus dem Arbeitsspeicher
- Adressierung der Datenpakete ähnlich dem Verfahren der FPU

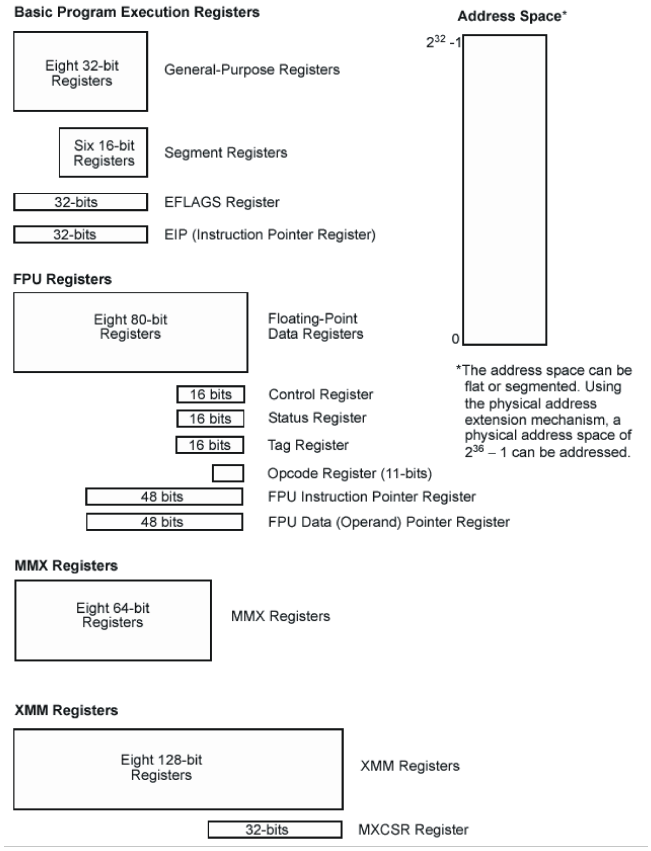



Bild 6.4 Registersatz des Pentiums

## Verwaltung des Arbeitsspeichers

- Implementation einer tabellengestützten Segmentation
  - Segmentation arbeitet als Blockübersetzung von Adressen
  - durch Eintrag in entsprechende Tabellen kann das Betriebssystem ein Programm an eine beliebige Stelle im Arbeitsspeicher schieben
  - die Segmentierung ist im 'protected mode' (Standardmode für Windows und Linux) immer aktiv

- Implementation einer virtuellen Adressierung
  - beim Pentium in der 'paging unit' realisiert
  - die 'paging unit' arbeitet gleichfalls tabellengestützt
  - prinzipiell ist die 'paging unit' abschaltbar (jedoch nicht sinnvoll, da sowohl Windows als auch Linux die 'paging unit' voraussetzen)
  - 'paging unit' realisiert eine weitere Adressumsetzung
    - Adressumsetzung ist in 'pages' (a 4 Kbyte) organisiert

# Kapitel 7

# Assembler und Hochsprache

## 7.1 Einordnung der Assemblerprogrammierung

Die Programmierung in Assembler eröffnet den ungehinderten Zugang zu jedem Prozessor. Dies hört sich zunächst sehr vielversprechend an, ist jedoch bei näherem Hinsehen eher zweischneidig. Durch die nicht begrenzten Zugriffsmöglichkeiten auf einen Prozessor entstehen auch viele Quellen für Fehler.
Natürlich ist die Programmierung in Assembler prädestiniert dafür, einen Prozessor in allen Einzelheiten kennen zulernen. Dies ist der Hauptgrund für die Beschäftigung mit der Assemblerprogrammierung in dieser Vorlesung. Tatsächlich wird man jedoch kaum in die Verlegenheit kommen, größere Programmteile für einen Pentium in Assembler zu programmieren. Neben den vermehrten Fehlermöglichkeiten ist auch die mangelnde Wartungsmöglichkeit von Assemblerprogrammen und die fehlende Portierbarkeit der Grund auf diese Art von Programmen mehr oder weniger zu verzichten.
Bei einem Wechsel von einem Pentium zu anderen Prozessoren, insbesondere zu Mikrocontrollern ändert sich das Bild. In derartigen Umgebungen kommen Assemblerprogramme durchaus noch zum Einsatz. Jedoch wird man auch hier kaum ausschließlich Programme in Assember finden, sondern meist eine Kombination von C- und Assemberprogrammen. Dies verleitet uns dazu, in dieser Vorlesung einen ähnlichen Ansatz zu verfolgen. C-Programmen stellen das Gerüst einer Anwendung dar und für spezielle Teile werden Assemblerroutinen eingesetzt, die dann von dem C-Programm aufgerufen werden. Die Kombination von C- und Assemblerprogramm ist insbesondere bei der Parameterübergaben, die praktisch die Schnittstelle zwischen diesen beiden Arten von Programmen darstellen, von besonderer Bedeutung, weshalb wir dieser Thematik ein besonderes Kapitel widmen wollen.

**Definition 7.1** Ein *Programm* ist eine Folge von Einzelbefehlen (Instruktionen), die von einem Rechner unter Lenkung eines Steuerwerkes nacheinander ausgeführt werden.

- Klassifikation von Programmiersprachen
  - Problemorientierte Programmiersprache (z. B. C)
  - Maschienenorientierte Programmierung (z. B. Assembler- Programmierung)
  - eigentlicher Maschinenbefehl (in Binärcode vorliegende Anweisung, die unmittelbar vom Steuerwerk dekodiert und ausgeführt wird)

- Verwendung von Programmiersprachen

    - Problemorientierte Programmiersprache C
    - bei nahezu jedem Prozessor (incl. Controller, Signalprozessor) anzutreffen
    - manchmal als 'moderner Assembler' gekennzeichnet
    - Hardwareeigenschaften des Prozessors versteckt

- Assembler
    - Hardwareeingenschaften des Prozessors liegen offen
    - Einsatz in speziellen (zeitkritische und speicherkritischen) Anwendungen
    - manchmal der 'kleinste gemeinsame Nenner' bei Anwendungen mit mehreren Programmiersprachen/Bibliotheken
    - bei Echtzeit- und Treiberentwicklungen häufiger anzutreffen

- Kombination C/Assembler
    - reine Assemblerentwicklungen relativ selten anzutreffen (zu fehlerträchtig, wartungsunfreundlich)
    - Kombination von Assemblermodulen und C-Modulen
        - Verwendung von 'Inline-Assembler' (wenig wartungsfreundlich)
        - Aufruf von Assemblerprogrammen aus C-Modulen heraus
        - Parameterübergaben müssen geklärt sein

## 7.2 Parameterübergabe

### Kombination von Assembler- und C-Programmen

- viele C-Compiler bieten die Möglichkeit direkt Assemblerbefehle ins Programm einzubauen
- Verwendung von 'Inline'-Assemblern
- Vorteil: einfach zu implementieren (kein eigentlicher Assembler notwendig)
- Nachteil: C-Programm verliert sein Plattform-Unabhängigkeit (Nachteile der Assemblerprogrammierung in C-Programme eingeschleppt)

- Trennung in C- und Assemblermodule
    - getrennte Programmierung der C- und der Assemblermodule
    - getrennte Pflege der unterschiedlichen Module
    - bei Plattform-Wechsel nur Neuprogrammierung der Assemblermodule notwendig
    - Aber: Verfahren der Parameterübergabe müssen klar definiert sein (Parameter passing conventions)
    - Verfahren der Parameterübergabe bei verschiedenen Compilern ähnlich, jedoch nicht gleich (kein genormtes Verfahren)

- die meisten C-Compiler bieten zwei Möglichkeiten der Kombination von C- und Assembler-Modulen
  - Aufruf eines Assemblerprogramms aus einer C-Funktion heraus (häufigste Anwendung)
  - Aufruf von C-Funktionen (meist aus einer C-Bibliothek) von einem Assemblerprogramm aus (selten)

- Spezifikation der Schnittstelle
  - nicht einheitlich definiert (parameter passing or C-calling conventions)
  - ähnliches Vorgehen vieler C-Compiler (Beispiel gcc)
    - Assemblerprogramm als Funktion konzipieren
    - der Rückgabewert wird immer im Accumulator abgelegt (8-, 16- oder 32Bit je nach Spezifikation des Rückgabetyps)
    - wird ein float oder double zurückgegeben so ist er im ST(0) abgelegt (Accumulator der FPU)

- Parameterübergabe mittels Stack (bei Intel-Mikroprozessoren fast immer anzutreffen; nicht jedoch beim PowerPC)
- Reihenfolge der Parameter mit einem 'pragma' (Borland, MSC) oder einem Macro (gcc) festgelegt

**Beispiel 7.1**

Prototype der Funktion
int ass_f (int, int*) _attribute_((cdecl));

Verwendung der Funktion (z.B)
error = ass_f(12, &i);

Parameter auf dem Stack:
12 -> Wertparameter (call by value)
&i -> Wertparameter Adresse des Integer i (call by reference)
(minimale Wortbreite bei Zugriff auf Stack = 16 Bit!)

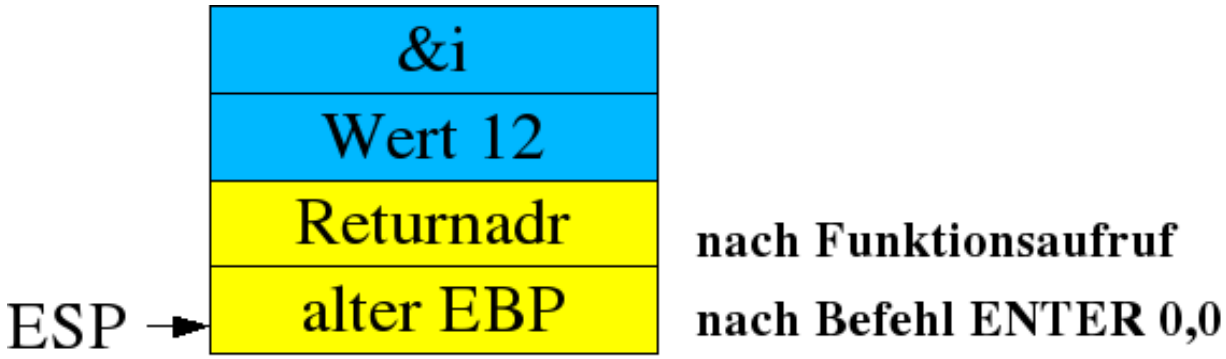



Bild 7.1 Aufbau des Stacks nach Funktionsaufruf

- Zugriff auf Parameter mit Register EBP und Offsetadressierung
  - z. B. MOV EBX,[EBP+8]

- Rückgabewert in Register EAX
- Verlassen der Funktion mit Befehl Leave (companion to Enter)

### Weitere Randbedingungen der Schnittstelle

- die quasi-Adress-Register EBX, ESI, EDI und EBP dürfen nicht verändert werden (Verwendung in der Assemblerfuktion bedeutet: Sichern des alten Inhaltes)
- der Name der Assemblerfunktion muss als 'Global' bekannt gemacht werden (und in C mit einem führenden Unterstrich angesprochen)

**Beispiel 7.2** int ass_f (int, int*) _attribute_((cdecl));

Assemblerprogramm

```
segment     .text        ;Beginn der Funktion
global ass_f
ass_f:                   ;Label der Funktion
            enter 0,0
            .
            .
            mov eax,0    ;Returnvalue = 0, alles o.k.
            leave
            ret
```

- die Segmentvorgaben dürfen nicht verändert werde
- globale Variable, Static-Variable der C-Funktion sind in der Assemblerfunktion sichtbar (wieder mit führendem Unterstrich)

## 7.3 Werkzeuge zur Programmentwicklung

- Entwicklung von Assemblerprogramme für Mikroprozessoren erfolgt meist in den Zielsystemen
- Assemblerprogramme für Mikrocontroller oder Signalprozessoren werden meist als 'Cross'-Entwicklungen getätigt (Entwicklungssystem und Zielsystem differieren)

### Entwicklung im Zielsystem

- 'Klassische' Werkzeuge mit Entwicklungssystem sind z.B. TASM (Turbo Assembler, Borland) MASM (Microsoft Assembler)
- Entwicklungssystem umfaßt Editoren, Assembler, C-Compiler, Linker, Source Level-Debugger und Make-Utility (Steuerung der Übersetzungsschritte)
- Vorhandensein des entsprechenden Betriebssystems

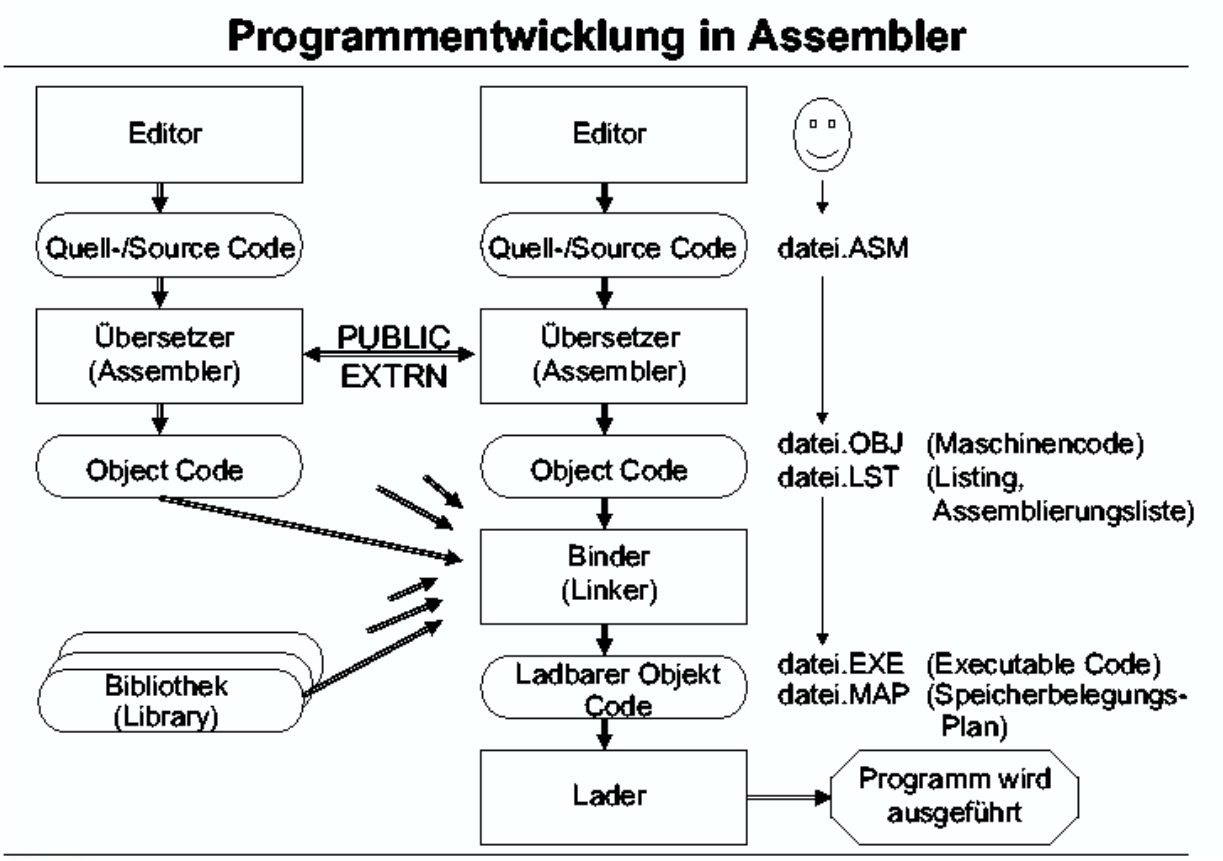



Bild 7.2 Ablauf einer Programmentwicklung

## Entwicklung unter Linux

- Verwendung des Assemblers NASM (Netwide Assembler)
  - ähnliche Notation wie TASM oder MASM (Windows)
- Einsatz von gcc (ld) als C-Compiler (Linker)
- als Debugger kommt gdb bzw. ddd (graphical front end) zum Einsatz
- Steuerung der Übersetzungsschritte mit 'make' und entsprechendem 'Makefile'

**Beispiel 7.3** Programmgerüst (NASM)

```
%include „asm_io.inc”          ;Einfache I/O-Funktionen
segment        .data           ;initialisierte Daten

segment        .bss            ;nichtinitialisierte Daten

segment        .text           ;Programmbereich
global         _asm_main       ;Start-Label des Programms
_asm_main:
               enter 0,0       ;Stack und EBP vorbereiten
               pusha           ;alle Register retten (muss nicht immer sein!)
               .
               .
               popa            ;alle Register restaurieren
               mov eax,0       ;Rückgabewert der Funktion
               leave
               ret
```

# Kapitel 8

# Übungen

## 8.1 Aufgaben zum Kapitel Einleitung

**Aufgabe 1**

Die unterschiedlichen Prozessoren lassen sich in drei große Gattungen einteilen.

- Stellen Sie die typischen Einsatzbereiche von Mikroprozessoren, Mikrocontrollern und Signalprozessoren dar!
- Inwieweit ist in einer Applikation ein Wechsel zwischen den verschiedenen Gattungen sinnvoll?

**Aufgabe 2**

Ein wesentlicher Gestaltungsgrundsatz für den Entwurf von Prozessoren ist deren Orthogonalität.

- Erläutern Sie die Orthogonalität an einem bestimmten Beispiel!
- Stellen Sie dar, was die Orthogonalität eines bestimmten Prozessors für dessen Programmierung bewirkt?

**Aufgabe 3**

Für die Entwicklung von Programmen für Mikroprozessoren stehen neben einer Vielzahl von Compilern und Assemblern auch Betriebssysteme bereit.

- Warum sind die erwähnten Entwicklungswerkzeuge vorzugsweise bei Mikroprozessoren zu finden?
- Mit welchen Einschränkungen muss man rechnen, wenn Programme für Mikrocontroller entwickelt werden sollen?

**Aufgabe 4**

Zur Verarbeitung von gebrochen rationalen Zahlen existieren verschiedene Möglichkeiten die bei der Verwendung eines Mikroprozessors oder eines Mikrocontrollers meist unterschiedlich zum Einsatz kommen.

- Stellen Sie dar, auf welchen Prozessoren (Mikroprozessoren bzw. Mikrocontrollern) welche Darstellung von gebrochen rationalen Zahlen üblich ist!
- Welche Besonderheiten sind in Verbindung mit den vier Grundrechenarten zu beachten, wenn die Wortbreite für die Zahlendarstellung nicht verändert werden darf?

**Aufgabe 5**

Um gebrochen rationale Zahlen darzustellen existieren in Prozessoren unterschiedliche M?glichkeiten.

- Stellen Sie dar,wie eine gebrochen rationale Zahl in einem Prozessor mit der Wortbreite von 32 Bit dargestellt werden kann und erläutern Sie die entsprechenden Vor- und Nachteile!
- Wie lassen sich die Zahlen +0,375 und -0,375 in den einzelnen Formaten darstellen? (Geben Sie bitte die entsprechende Bin?r- oder hexadezimale Kodierung an!)

## 8.2 Aufgaben zu den Kapiteln Speicher

**Aufgabe 6**

Stellen Sie die Unterschiede zwischen einem SRAM und einem DRAM dar und erläutern Sie in welchen Anwendungen eher der eine oder der andere Speichertyp anzutreffen ist!

**Aufgabe 7**

Der Arbeitsspeicher eines Rechnersystems ist neben dem Prozessor kennzeichnend für die Leistungsfähigkeit des Gesamtsystems.

- Stellen Sie den Zusammenhang zwischen Adressregister und möglicher Größe des Arbeitsspeichers dar!
- Worin unterscheiden sich eine lineare Adressierung von einer segmentierten und einer virtuellen Adressierung?
- Wann ist die Verwendung von Schaltungen für die Adressdekodierung sinnvoll?

**Aufgabe 8**

Welche besondere Funktion hat ein Cache-Speicher in einem Rechnersystem und aus welchen Speichertypen ist er aufgebaut?

## 8.3 Aufgaben Bussystem und Schnittstellen

**Aufgabe 9**

Stellen Sie den Zusammenhang wischen der 'Baudrate' einer seriellen Schnittstelle und der zu erzielenden Datenübertragungsrate dar!

**Aufgabe 10**

Warum wird bei einer USB-Busschnittstelle eine Kodierung der zu transferierenden Daten (Bytes) vorgenommen während man bei der seriellen Schnittstelle auf eine Kodierung verzichtet?

## 8.4 Aufgaben zur 8086-Architektur

**Aufgabe 11**

In einem Rechnersystem, dass als Prozessor einen Pentium enthält ist ein Arbeitsspeicher von 1024 MByte vorgesehen.

- An welchen Adressen platziert ein Compiler oder Assembler die für eine Anwender-Programmierung benötigten Segmente?
- Wie bewerkstelligt das Betriebssystem eine Veränderung der vom Compiler oder vom Assembler vorgesehenen Platzierung des Anwenderprogramms in dem Arbeitsspeicher?
- Welche Möglichkeiten bietet der Pentium einem Betriebssystem um einem Anwenderprogramm, das größer als der zur Verfügung stehende Arbeitsspeicher ist, trotzdem zur Ausführung zu gelangen?

**Aufgabe 12**

Worin unterscheidet sich die Segmentierung eines Pentiums von der Segmentierung eines 8086-Prozessors?

**Aufgabe 13**

Wie wird beim Pentium die Adressierung von Daten (Schreib- und Lesezugriff) bewerkstelligt, die für Coprozessoren bestimmt?

# Kapitel 9

# Lösungen zu Übungsaufgaben

## 9.1 Aufgaben zum Kapitel Einleitung

**Aufgabe 1** Stellen Sie die typischen Einsatzbereiche von Mikroprozessoren, Mikrocontrollern und Signalprozessoren dar!

- Mikroprozessoren: General Purpose Prozessor für Anwendungen die Betriebssystemunterstützung, grafische Interfaces und Massenspeicher erfordern; Bereitstellung großer Rechenleistung und großer Arbeitsspeicher; Verwendung eines Motherboards erforderlich
- Mikrocontroller: Anwendungen mit dezidierten Algorithmen, die keine Betriebssystemunterst?tzung erfordern, Bereitstellung von analogen und digitalen Schnittstellen, geringe Rechenleistung erforderlich; Implementation von funktionsfähigen Gesamtsystemen mit geringem Aufwand
- Signalprozessor: Gleicher Anwendungsbereich wie Mikrocontroller aber mehr Rechenleistung

Inwieweit ist in einer Applikation ein Wechsel zwischen den verschiedenen Gattungen sinnvoll?

- Nur Wechsel zwischen Mikrocontrollern und Signalprozessoren sinnvoll! Gleicher Anwendungsbereich der beiden Prozessorgattungen.

**Aufgabe 2** Erläutern Sie die Orthogonalität an einem bestimmten Beispiel!

- Funktionseinheiten von Prozessoren unabhängig voneinander konzipieren. Beispiel ist die Adressierungseinheit, die von der FPU und von der Integer-Einheit verwandt wird.

Stellen Sie dar, was die Orthogonalität eines bestimmten Prozessors für dessen Programmierung bewirkt?

- Programmierung und Compilerbau gestalten sich einfacher. Keine Sonderbehandlung bei bestimmten Befehlen. Programme werden kleiner.

**Aufgabe 3** Warum sind die erwähnten Entwicklungswerkzeuge vorzugsweise bei Mikroprozessoren zu finden?

- Mikroprozessoren decken den weitaus größten Teil aller Prozessoranwendungen ab. Sie verfügen über die notwendigen Funktionen (Motherboard, Massenspeicher, GUI) um Entwicklungswerkzeuge (Compiler und Assembler) und insbesondere Betriebssysteme einsetzen zu k?nnen.

Mit welchen Einschränkungen muss man rechnen, wenn Programme für Mikrocontroller entwickelt werden sollen?

- Mikrocontroller verwenden keine Motherboards, Massenspeicher oder GUI zur Implementation eines lauffähigen Systems. Von daher wird kein Betriebssystem eingesetzt und die Entwicklungswerkzeug werden als Cross-Systeme verwandt.

**Aufgabe 4** Stellen Sie dar, auf welchen Prozessoren (Mikroprozessoren bzw. Mikrocontrollern) welche Darstellung von gebrochen rationalen Zahlen üblich ist!

- Mikroprozessoren verfügen meist über eine FPU. Die gebrochen rationalen Zahlen werden als Floationg Point-Zahlen dargestellt. Die Verarbeitung von gebrochen rationalen Zahlen wird von der FPU übernommen. Eine Implementation von Fix Point-Zahlen ist nicht notwendig. Bei Mikrocontrollern fehlt meist die FPU. Die Implementation von Fix Point-Zahlen ist also die einzig sinnvolle Möglichkeit mit gebrochenrationalen Zahlen zu operieren.

Welche Besonderheiten sind in Verbindung mit den vier Grundrechenarten zu beachten, wenn die Wortbreite für die Zahlendarstellung nicht verändert werden darf?

- Floating Point-Zahlen werden nach jeder Rechenoperation normiert; die Wortbreite ändert sich also nicht! Fix Point-Zahlen können die Wortbreite nach jeder Rechenoperation verändern, wenn die Genauigkeit beibehalten werden soll. Muss die Wortbreite eingehalten werden, so ist unter Verzicht der Genauigkeit eine Rundung und oder Begrenzung der Wertebereiche vorzunehmen.

**Aufgabe 5** Stellen Sie dar,wie eine gebrochen rationale Zahl in einem Prozessor mit der Wortbreite von 32 Bit dargestellt werden kann und erläutern Sie die entsprechenden Vor- und Nachteile!

- a) als Floating-Point-Zahl mit automatischer Normierung (Vorteil). Nachteil: Verwendung einer FPU erforderlich oder sehr rechenzeitintensiv. Darstellung von Sonderzeichen wie z. B. 0 ist notwendig
- b) als Fix-Point-Zahl jedoch ohne automatische Normierung (Nachteil). Vorteilhaft ist, dass keine FPU erforderlich ist und die Zahl 0 direkt darstellbar ist.

Wie lassen sich die Zahlen +0,375 und -0,375 in den einzelnen Formaten darstellen? (Geben Sie bitte die entsprechende Bin?r- oder hexadezimale Kodierung an!)

- a) Floating-Point-Zahl +0,375 = (+) 1,5 * 2 exp(2) = 0(Vorzeichen) 1000..(Mantisse) 00000010 (Exponent) -0,375 = (-) 1,5 * 2 exp(2) = 1(Vorzeichen) 1000..(Mantisse) 00000010 (Exponent)
- b) Fix-Point-Zahl +0,375 = 00006000H -0,375 = 0FFFFA000H (32 Bit Format)

## 9.2 Aufgaben zu den Kapiteln Speicher

**Aufgabe 6** Stellen Sie die Unterschiede zwischen einem SRAM und einem DRAM dar und erläutern Sie in welchen Anwendungen eher der eine oder der andere Speichertyp anzutreffen ist!

- SRAM = statisches RAM, Speicherzelle enthält Flipflop, kein Refresh notwendig, Adresse wird unmittelbar an den Speicherchip gelegt, sehr schneller Speicherzugriff, in Mikrocontroller- oder Signalprozessor-Anwendungen als externer Speicher zu finden

- DRAM = dynamisches RAM, Speicherstelle ist ein Kondensator, Refresh erforderlich, Einteilung der Adresse in eine Spalten- und Zeilen-Adresse notwendig, langsamerer Speicherzugriff; in Mikroprozessor-Anwendungen als Arbeitsspeicher zu finden

**Aufgabe 7** Stellen Sie den Zusammenhang zwischen Adressregister und möglicher Größe des Arbeitsspeichers dar!

- Die Wortbreite des Adressregisters muss als exponential-Ausdruck zur Basis 2 mindestens den Adressumfang des Arbeitsspeichers umfassen

Worin unterscheiden sich eine lineare Adressierung von einer segmentierten und einer virtuellen Adressierung?

- Bei der linearen Adressierung entspricht die Adresse im Programm der Adresse auf dem Adressbus.
- Bei der segmentierten Adressierung wird ein Segmentvorsatz verwandt, der eine Vergrößerung des Adressraums oder eine Verschiebung des Segments im Adreßraum und/oder eine Vergabe von Zugriffsrechten bewirkt.
- Die virtuelle Adressierung bewirkt eine Adressumsetzung in kleineren Blöcken als die Segmentierung. Typische Blockgrößen (pages) sind 4 KByte.

Wann ist die Verwendung von Schaltungen für die Adressdekodierung sinnvoll?

- Immer dann wenn der externe Speicher kleiner ist als der Adreßraum des Prozessors.

**Aufgabe 8** Welche besondere Funktion hat ein Cache-Speicher in einem Rechnersystem und aus welchen Speichertypen ist er aufgebaut?

- Ein Cachespeicher dient als Pufferspeicher zwischen dem Prozessor und dem eigentlichen Arbeistspeicher und entkoppelt den 'schnellen' Prozessor (kurze Speicherzugriffszeiten) vom 'langsamen' Arbeitsspeicher (längere Speicherzugriffszeiten). Er ist immer aus SRAM-Speicher aufgebaut.

## 9.3 Aufgaben Bussystem und Schnittstellen

**Aufgabe 9** Stellen Sie den Zusammenhang wischen der 'Baudrate' einer seriellen Schnittstelle und der zu erzielenden Datenübertragungsrate dar!

- Die 'Baudrate' einer seriellen Schnittstellen gibt die Übertragungsrate in Bit/s an. Da in dem der Schnittstelle zugehörigem Protokoll die Anzahl der zu übertragenden Bits pro Byte (Startbit, Anzahl Bits pro Byte, 1 oder 2 Stoppbits, eventuell 1 Parity-Bit) festgelegt sein muss gibt dieses 'Protokoll' den gesuchten Zusammenhang zwischen der 'Baudrate' und der erzielbaren Datenübertragungsrate vor.

**Aufgabe 10** Warum wird bei einer USB-Busschnittstelle eine Kodierung der zu transferierenden Daten (Bytes) vorgenommen während man bei der seriellen Schnittstelle auf eine Kodierung verzichtet?

- Der Grund für die Kodierung der Daten bei einer USB-Schnittstelle liegt in der Notwendigkeit, aus den transferierten Daten das Taktsignal, mit dem die Daten gesendet wurden, zu rekonstruieren. Dies gelingt nur, wenn die Daten besonders kodiert werden (NRZI-Code mit Bit-Stuffing). Die serielle Schittstelle arbeitet mit zwei unterschiedlichen Taktgeneratoren, die zwar über die gleiche 'Baudrate' verfügen müssen, aber es erfolgt keine Taktrückgewinnung aus den transfertierten Daten.

## 9.4 Aufgaben zur 8086-Architektur

**Aufgabe 11** An welchen Adressen platziert ein Compiler oder Assembler die für eine Anwender-Programmierung benötigten Segmente?

- Compiler oder Assembler benutzen für jedes Anwenderprogramm ein entsprechendes Programmsegment (Codesegment), ein Datensegment und ein Stacksegment. Das Programm- und Datensegment werden an der Adresse 0 plaziert und die Stacksegmente an der maximalen Adresse (0xFFFFFFFF).

Wie bewerkstelligt das Betriebssystem eine Veränderung der vom Compiler oder vom Assembler vorgesehenen Platzierung des Anwenderprogramms in dem Arbeitsspeicher?

- Mit Hilfe der Segmentvorgaben (CS-Selector für Codesegment, DS-, ES-, FS-, oder GS-Selector fürs Datensegment, SS-Selector für Stacksegment) und der automatisch durchgeführten Segmentierung werden die entsprechenden Segmente an beliebiger Stelle im Arbeistspeicher verschoben.

Welche Möglichkeiten bietet der Pentium einem Betriebssystem um einem Anwenderprogramm, das größer als der zur Verfügung stehende Arbeitsspeicher ist, trotzdem zur Ausführung zu gelangen?

- Mit Hilfe der 'Paging Unit' wird ein Anwenderprogramm in so viele Pages zerlegt wie es der Größe des Anwenderprogrammes entspricht. Die jeweils benötigten Pages werden in den Arbeistsspeicher geladen und nicht benötigte Pages ausgelagert.

**Aufgabe 12** Worin unterscheidet sich die Segmentierung eines Pentiums von der Segmentierung eines 8086-Prozessors?

- Die Segmentierung eines Pentiums arbeitet tabellengestützt. Dies bedeutet, dass der jeweilige Selektor (CS-Selektor für Programmsegment, DS-, ES-, FS- bzw. der GS-Selektor fürs Datensegment und der SS-Selektor fürs Stacksegment) auf Einträge in Tabellen (Descriptortable) zeigen und diese Einträge für die Adressierung verwendet werden. Das Betriebssystem verwaltet die entsprechenden Einträge. Beim 8086-Prozessor werden keine Tabellen verwandt sondern die Adressierung erfolgt nach dem Algorithmus 'Eintrag in dem entsprechenden Segmentregister x 16 plus Eintrag aus dem Adressregister'. Das Betriebssystem kann logischerweise den Algoritmus auch nicht verändern.

**Aufgabe 13** Wie wird beim Pentium die Adressierung von Daten (Schreib- und Lesezugriff) bewerkstelligt, die für Coprozessoren bestimmt?

- Daten für die Coprozessoren (FPU = Floating Point Unit, MMX = Multi media extension, SSE single instruction multiple data streaming extension) stammen grundsätzlich aus dem externen Arbeitsspeicher. Die Adressierung erfolgt wie ein normaler Zugriff der CPU auf den Arbeitsspeicher. Das bedeutet, dass die CPU-Register die Adressregister der Coprozessoren bilden. Lediglich die Größe des Datums, das transferiert werden soll muss angegeben werden (DWORD, QWORD, DQWORD = 32 Bit, 64 Bit 128Bit) Innerhalb des Coprozessors ist auch ein Register zu Register-Transfer möglich.

# Literaturverzeichnis


[1] Wikipedia – Die freie Enzyklopädie. www.wikipedia.org.

[2] Wikibook – Online Biblothek. http://de.wikibook.org/Assembler_(80x86_Prozessor)-Programmierung

[3] T.E. Podschun *Das Assembler-Buch*. Addision-Wesley, 1998. Assemblerprogrammierung von 8086 - Pentium.

[4] J.L. Hennessy and D.A. Patterson, *Computer Architecture*. Morgan Kaufmann Publishers, 2003. Die 'Bibel' für die Rechnerarchitektur.

[5] *Netwide Assembler*. http://sourceforge.net/projects/nasm